Li *et al. Cybersecurity* (2026) 9:150
https://doi.org/10.1186/s42400-026-00571-w

Cybersecurity

SURVEY

Open Access

A survey of optimization techniques for bootstrapping algorithms in FHE

Tianyu Li[1,2], Zhiwei Wang[1,2], Lutan Zhao[1,2*], Ruida Wang[1,2], Qingyun Niu[1,2], Xianhui Lu[1,2], Dan Meng[1,2] and Rui Hou[1,2*]

Abstract

Fully Homomorphic Encryption (FHE) enables arbitrary computation on encrypted data without decryption, making it a cornerstone of privacy-preserving outsourcing, such as cloud computing. However, homomorphic operations cause ciphertext noise to grow until decryption fails. The efficient solution is bootstrapping, which refreshes the noise in FHE ciphertexts to sustain arbitrary deep homomorphic evaluation. But in practice, bootstrapping consumes over 50% of total execution time, posing a serious obstacle to FHE adoption. This paper presents a systematic survey of FHE bootstrapping algorithms and their optimizations. We organize existing works into three main paradigms: word-wise bootstrapping for BGV, BFV, and CKKS schemes; bit-wise bootstrapping for FHEW and TFHE schemes; and hybrid bootstrapping, which leverages both word-wise schemes and bit-wise schemes. We analyze the evolution of crucial techniques, highlight latest advances in reducing latency, enhancing parallelism, and controlling noise growth, and compare the advantages and limitations of different schemes. Finally, we discuss emerging research trends.

Keywords Fully homomorphic encryption, Bootstrapping, Word-wise, Bit-wise

Introduction

The rapid advancement of cloud computing, big data, and artificial intelligence has increased reliance on sensitive user data, raising urgent global concerns over privacy protection. Traditional encryption requires key sharing or decryption for collaborative computation, thereby exposing plaintext and risking data leakage. Homomorphic Encryption (HE) addresses this challenge by allowing arithmetic operations on ciphertexts and producing correct results after decryption. In other words, HE makes data "usable yet invisible", ensuring end-to-end confidentiality throughout the entire data lifecycle—storage, transmission, and processing—fundamentally eliminating channels for sensitive information leakage.

Since the concept of HE was introduced in 1978 (Rivest et al. 1978), schemes have been categorized into Partially Homomorphic Encryption (PHE), Somewhat Homomorphic Encryption (SWHE), and Fully Homomorphic Encryption (FHE), according to the types and depths of supported ciphertext operations. PHE schemes support unlimited operations of a single type (Yiran et al. 2024) (e.g. multiplicative RSA Rivest et al. 1978 or additive Paillier 1999). Although functionally limited, the high efficiency of PHE has enabled widespread adoption in practice, especially Paillier (Fang and Qian 2021; Zhang et al. 2020; Anggriane et al. 2016; Yang and Järvinen 2018). SWHE supports both addition and multiplication, but only a bounded number of operations (e.g. the BGN Boneh et al. 2005 scheme allows just one multiplication). While SWHE laid the theoretical foundation for FHE, it suffers from noise accumulation: each homomorphic operation introduces additional error to

*Correspondence:
Lutan Zhao
zhaolutan@iie.ac.cn
Rui Hou
hourui@iie.ac.cn
[1] Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS, Beijing 100085, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing 101408, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

the ciphertext, and once this noise exceeds a threshold, decryption fails. Overcoming noise growth therefore has been the critical challenge for achieving FHE.

In 2009, Gentry introduced the bootstrapping (BTS) technique (Gentry 2009), which homomorphically evaluates the decryption circuit to lower the noise, thereby achieving support for arbitrary depth of multiplication. Bootstrapping has since become the cornerstone of FHE. Subsequently, FHE research has flourished along multiple parallel technical directions, with one major distinction being the granularity: plaintext vectors or individual bits (Fig. 1).

Word-wise FHE schemes: Representative schemes include BGV (Brakerski and Vaikuntanathan 2011; Brakerski et al. 2014), BFV (Brakerski 2012; Fan and Vercauteren 2012), and CKKS (Cheon et al. 2017, 2018). Word-wise FHE schemes are based on the Ring Learning with Errors (RLWE) problem and operate on vectors. BGV and BFV support exact integer arithmetic, while CKKS implements approximate arithmetic by treating noise as part of the plaintext, rendering it well-suited for machine learning and statistical analysis. These schemes provide large plaintext spaces and enable efficient batching, but bootstrapping remains a major performance bottleneck, typically requiring approximately 4–95 s depending on the scheme and parameter settings.

Bit-wise FHE schemes: Practical schemes include FHEW (Ducas and Micciancio 2015) and TFHE (Chillotti et al. 2020). Bit-wise FHE schemes introduce matrix-based ciphertexts to mitigate the vector-dimension blowup problem in word-wise schemes and achieve slower noise growth. They support programmable bootstrapping (PBS) for arbitrary Boolean functions with millisecond-level latency, offering unique advantages in security and flexibility compared to word-wise approaches. However, the lack of SIMD parallelism imposes significant limitations on their practical efficiency.

Hybrid FHE frameworks: Recent approaches combine the batching efficiency of word-wise schemes with the functional flexibility of bit-wise schemes. These architectures map the lookup table evaluation from bit-wise FHE into the SIMD infrastructure of word-wise schemes, enabling evaluating non-linear functions in parallel. Hybrid schemes preserve the word-wise FHE with thousands of slots while supporting rich function evaluation, offering a promising new direction for large-scale privacy-preserving computation.

Table 1 compares the four lattice-based FHE paradigms in terms of plaintext types, amortized bootstrapping latency, noise growth rate in homomorphic multiplication (HMult), and other critical metrics. Although FHE theoretically promises data confidentiality and offers provable security for outsourced tasks such as machine learning (Gilad-Bachrach et al. 2016), private set intersection (Gentry and Halevi 2019), and genome-wide association studies (Kim et al. 2020), its practical performance lags. FHE operations incur latency about 10,000× greater than their unencrypted counterparts (Samardzic and Sanchez 2024), with bootstrapping constituting the dominant overhead. Bootstrapping alone accounts for over 90% (Cheon et al. 2018; Ducas and Micciancio 2015; Halevi and Shoup 2021) of FHE computation (including addition, multiplication, key switching, and bootstrapping) and typically 50–80% (Najjar et al. 2025; Ohba et al. 2025) of runtime in applications such as federated learning and private inference. Consequently, thorough investigations and optimizations of bootstrapping techniques are pivotal to advancing FHE from theoretical constructs to practical applications.

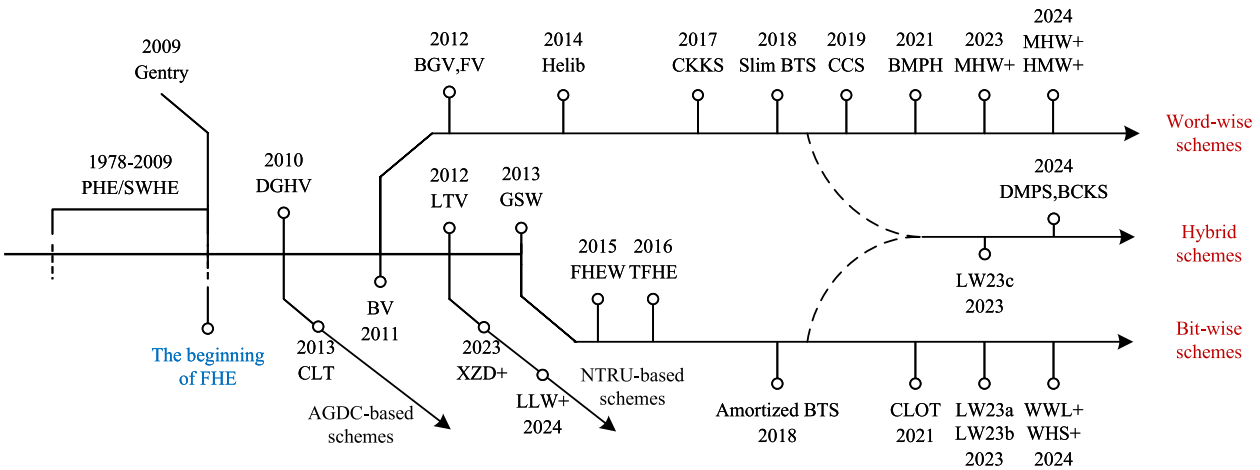


Fig. 1 Evolution of fully homomorphic encryption

Table 1 Comparison of four lattice-based FHE paradigms

Representative schemes	Hardness assumption	Granularity	Plaintext type	Ciphertext size	BTS time/bit	Noise growth in HMult (Yiran et al. 2024)
Gentry09	Ideal Coset Problem over ideal lattices	Bit-wise	Single bit {0,1}	$f(\lambda)$	31 min (Gentry and Halevi 2011)	$N^{\Omega(1)}$
BGV/BFV	RLWE	Word-wise	Integer vector	$2n\lceil\log_2 q\rceil$	10.19 s (Al Badawi et al. 2023)	$O(n^2)$
FHEW/TFHE	(R)LWE	Bit-wise	Single bit (→ single integer)	$N(k+1)^2 l$	13 ms (Chillotti et al. 2020)	$O(n)$
CKKS	RLWE	Word-wise	Floating-point vector	$2n\lceil\log_2 q\rceil$	1.1 s (Cheon et al. 2018)	$O(n^2)$

This survey focuses on bootstrapping techniques and optimizations. Section 2 reviews the general FHE workflow and the ciphertext formats used in word-wise and bit-wise schemes. Section 3 gives an overview of bootstrapping concepts and contrasts word-wise vs bit-wise vs hybrid approaches. Section 4 presents the basic bootstrapping procedure and optimization strategies for word-wise FHE, and establishes a theoretical framework to systematically analyze and compare bootstrapping algorithms according to their modular reduction methods. Section 5 discusses the bootstrapping algorithms and optimizations for bit-wise FHE, covering four subtypes: gate bootstrapping (GBS), programmable bootstrapping (PBS), circuit bootstrapping (CBS), and amortized bootstrapping (ABS). Section 6 addresses hybrid bootstrapping, detailing how lookup table evaluation in bit-wise schemes is integrated into the SIMD frameworks of word-wise schemes. Section 7 analyzes remaining challenges and future research opportunities. Finally, Section 8 concludes with a summary of the paper's contributions.

Table 2 Summary of notations

Symbol	Description
N	Degree of polynomial
n	LWE ciphertext dimension
k	GLWE ciphertext dimension
l	Gadget decomposition length
q	Ciphertext modulus
Q	Extended modulus for bootstrapping/key-switching
t	Plaintext modulus
Δ	Scaling factor
λ	Security level
β	Gadget base for decomposition
χ	Error distribution
σ	Standard deviation of error in secret key
E	Absolute upper bound of error
I_N	$N \times N$ identity matrix
$\mathbb{T}$	Torus (real numbers modulo 1)
BSK	Bootstrapping key
KSK	Key-switching key

Basic framework of FHE

This section introduces the general FHE workflow, and describes the ciphertext structures of word-wise and bit-wise schemes. In this paper, lowercase boldface letters denote polynomial, and Table 2 summarizes notation used below.

FHE workflow

An FHE scheme proceeds through the following five phases:

1) **Parameter selection:** $setup(\lambda, L) \rightarrow params$. A setup procedure chooses system parameters (dimension, moduli, error distribution, etc.) for target security level λ and circuit depth L.
2) **Key generation:** $KeyGen(params) \rightarrow pk, sk, evk$. Execute the key-generation algorithm to produce the public key pk, secret key sk, and evaluation keys evk, which include the key-switching key used after multiplication and automorphism and the bootstrapping key.
3) **Encryption:** $Enc(m_i, pk) \rightarrow ct_i, i = 1, 2, \ldots, r$. Encrypt each message m_i under pk to obtain ciphertext ct_i.
4) **Homomorphic evaluation:** $Eval(ct_1, ct_2, \ldots, ct_r, f, evk) \rightarrow ct_{ans}$. Evaluate a predefined function f on ciphertexts, and then use evaluation keys to perform key switching so that the result remains decryptable under sk.
5) **Decryption:** $Dec(ct_{ans}, sk) \rightarrow f(m_1, m_2, \ldots, m_r)$. Finally, apply the decryption algorithm with secret key to recover $f(m)$ in plaintext.

Each homomorphic addition or multiplication increases the noise in the ciphertext. Without bootstrapping,

once the noise exceeds a bound (depending on the modulus and error distribution), decryption fails. Bootstrapping is applied when noise is high, resetting the noise to a low level while preserving the encrypted data.

Ciphertext structures

Word-wise FHE schemes

A word-wise ciphertext is represented as a pair of polynomials: $ct = Enc(m) = (\boldsymbol{b}, \boldsymbol{a}) \in \mathbb{R}_q^2$, where $\boldsymbol{a}$ is a uniformly sampled random polynomial, and the calculation formula of $\boldsymbol{b}$ is related to the specific scheme:

$$b = \begin{cases} -\boldsymbol{a}\cdot\boldsymbol{s} + t\cdot\boldsymbol{e} + \boldsymbol{m}, & \text{(BGV)} \\ -\boldsymbol{a}\cdot\boldsymbol{s} + \boldsymbol{e} + \Delta\boldsymbol{m},\ \Delta = \lfloor q/t \rfloor & \text{(BFV)} \\ -\boldsymbol{a}\cdot\boldsymbol{s} + \boldsymbol{e} + \Delta\boldsymbol{m},\ \Delta = 2^p & \text{(CKKS)} \end{cases}$$

In CKKS, p determines floating-point precision and can be adjusted according to the operating characteristics—bootstrapping typically requires at least 55 bits of precision, whereas ordinary homomorphic addition and multiplication tolerate 30–45 bits.

Bit-wise FHE schemes

LWE ciphertext: An LWE ciphertext encrypting a bit or small message m has the form: $LWE_s^{t/q}(m, E) = (a_0, \ldots, a_{n-1}, b) \in \mathbb{Z}_q^{n+1}$, where $\boldsymbol{s}$ is the secret key, with error bounded by E. Given plaintext m and random error e, $b = (\sum a_i s_i + mq/t + e) \pmod q$. The corresponding decryption operation is $m' = \left\lceil t(b - \boldsymbol{a}\cdot\boldsymbol{s})/q \right\rfloor \bmod t$, where $\lceil\cdot\rfloor$ represents center rounding that maps real numbers to the nearest integer.

GLWE ciphertext: Generalized LWE (GLWE) replaces scalars with length-N polynomials. A GLWE ciphertext is represented as $GLWE_s^{t/q}(\Delta\boldsymbol{m}) = (\boldsymbol{a}_0, \ldots, \boldsymbol{a}_{k-1}, \boldsymbol{b}) \subseteq \mathbb{R}_q^{k+1}$, where $\Delta = \lfloor q/t \rfloor$ and $\boldsymbol{b} = \sum_{i=0}^{k-1} \boldsymbol{a}_i\boldsymbol{s}_i + \Delta\boldsymbol{m} + \boldsymbol{e}$. When k equals 1, GLWE is the usual RLWE ciphertext; and LWE is the special case of GLWE when $k = n$ and each polynomial dimension N equals 1.

GGSW ciphertext: GGSW is a matrix extension of GLWE, expressed as a $(k+1)l \times (k+1)$ polynomial matrix (as shown in Fig. 2). Specifically, each GLWE ciphertext is a $k+1$-dimensional polynomial, and GGSW uses β as factorization basis to decompose the message down to l levels. This matrix contains $k+1$ independent GLWE ciphertexts, encrypting $\boldsymbol{s_0 m}, \boldsymbol{s_1 m}, \ldots, \boldsymbol{s_{k-1} m}, \boldsymbol{m}$ respectively. RGSW is the special case when $k = 1$. This structure provides necessary support for bootstrapping by encrypting the key-message product.

Overview of FHE bootstrapping technology

Bootstrapping in fully homomorphic encryption conceptually has two main steps: (1) **Scalar product:** Compute the dot product between the ciphertext and the secret key, i.e. $b - \boldsymbol{as}$; (2) **Rounding:** Reduce the scalar product result from the ciphertext modulus to the plaintext modulus, followed by centered rounding to recover the message. In bit-wise schemes, the dominant computational cost lies in computing the scalar product $\langle \boldsymbol{a}, \boldsymbol{s} \rangle$, since the required modular arithmetic is handled efficiently by mapping the result onto the cyclotomic ring. In contrast, word-wise schemes face their primary bottleneck in the modular reduction, which is nonlinear and not natively supported by the underlying FHE primitive.

We categorize existing bootstrapping algorithms into three families according to their foundational approach,

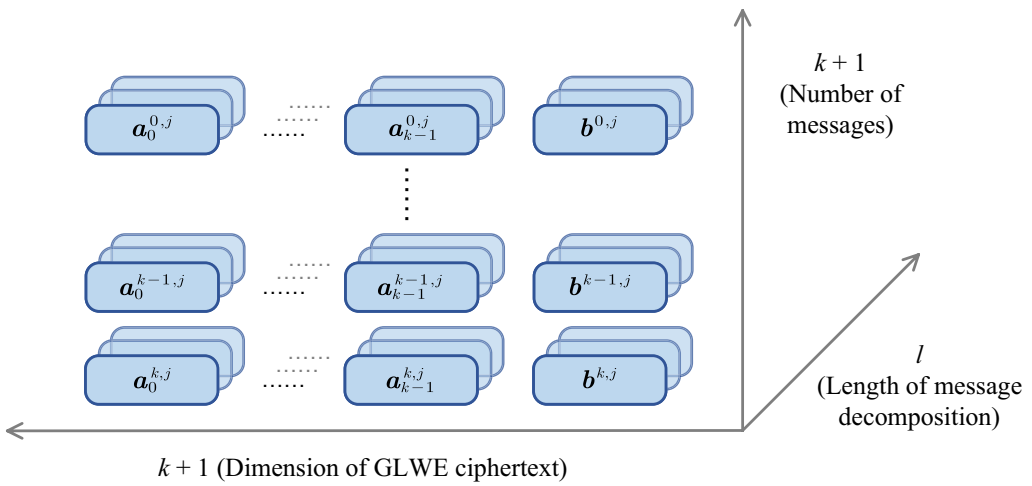


Fig. 2 Schematic of the GGSW ciphertext structure (Chillotti 2022)

and the comparison of key characteristics and performance metrics is summarized in Table 3. Since bootstrapping latency is highly sensitive to the chosen security level, parameter set, and (for CKKS) the target precision, the reported values should be interpreted as representative ranges indicating the order of magnitude. Ciphertext-size entries are derived under standard 128-bit-security parameterizations following the Homomorphic Encryption Standard (HES HomomorphicEncryption.org 2018). For RLWE-based (word-wise) schemes, the ciphertext size is $2N \cdot \lceil \log_2 Q \rceil$ bits, with practical parameters typically ranging from small-scale configurations ($N = 2048$, $\log_2 Q \approx 56$) to deep-circuit configurations ($N = 32768$, $\log_2 Q \approx 880$). In contrast, LWE-based (bit-wise) ciphertexts have size $(n+1) \cdot \lceil \log_2 q \rceil$ bits, where representative parameter choices span from ternary-secret settings ($n \approx 630$, $\log_2 q \approx 32$) to HES-compliant uniform-secret settings ($n = 1024$, $\log_2 q \approx 32$).

In summary, word-wise schemes generally incur larger ciphertext size because their noise grows more rapidly than bit-wise schemes; as a result, they require larger moduli and stronger security assumptions (relying on lattice-hardness with super-polynomial approximation factors, whereas bit-wise schemes need only polynomial-level factors). Nonetheless, the SIMD batching inherent in word-wise constructions provides a substantial practical advantage. In particular, hybrid bootstrapping built on word-wise constructions achieves higher slot parallelism and lower amortized cost per slot than purely bit-wise amortized bootstrapping (see Table 6).

Word-wise FHE bootstrapping algorithms

Bootstrapping in BGV scheme

The original BGV bootstrapping (Gentry et al. 2012) works with a special modulus $q = 2^r + 1$ for a positive integer r. By choosing q this way, one can recover the plaintext bit by adding a constant and extracting specific integer bits, instead of doing full modular reduction via binary circuit.

Property 1. Let $q = 2^r + 1$. For any integer z satisfying $|z| < \frac{q^2}{4} - q$ and $[z]_q \in (-\frac{q}{4}, \frac{q}{4})$, the value $[[z]_q]_2$ equals the XOR of the least significant bit and the r-th bit of z', where $z' = z + \frac{q^2-1}{4}$.

Based on this insight, the BGV bootstrapping algorithm proceeds as follows:

1) **Ciphertext Preprocessing:** Given a ciphertext $\boldsymbol{ct} = (\boldsymbol{c}_0, \boldsymbol{c}_1)$, add the constant $\boldsymbol{c^*} = (2^{r-1}, 0)$ to produce a new ciphertext $\boldsymbol{c'}$ with modulus $2^r + 1$.
2) **Homomorphic Scalar Product:** Compute the scalar product of ciphertext $\boldsymbol{c'}$ and the bootstrapping key BSK, yielding the ciphertext of polynomial Z'.
3) **Coefficient-to-Slot:** Apply the inverse discrete Fourier transform to convert the coefficients of Z' into plaintext slots, enabling subsequent parallel bit extraction.
4) **Bit Extraction:** Iteratively apply the mapping from Property 1 to shift each target bit into the least significant position. Specifically, Gentry et al. constructs an integer sequence

Table 3 Comparison of technical approaches to bootstrapping in FHE

BTS approach		Latency	Slots num	Func types	Cipher size	Noise	Core Ops
Word-wise BTS	BGV BFV	22–95 s (Liu and Wang 2024)	Typically 64 to 2048, $\leq N$	Identity function	28 KB–6.88 MB	Refresh noise	Digit extraction
	CKKS	3.9–34.8 s (Bossuat et al. 2021; Seal 2023)	Usually tens of thousands, $\leq N/2$	Continuous functions on an interval, polynomial-approximable	28 KB–6.88 MB	Restore level while introducing approximation error	Polynomial approximation modulo
Bit-wise BTS	Gate BTS	<10 ms	1	Boolean Gate	2.5 KB–4 KB	Refresh noise	Blind rotation
	Programmable BTS	9.5–6400 ms	1	arbitrary function	2.5 KB-4 KB	Refresh noise	Blind Rotation
	Circuit BTS	88.6–877 ms	1	–	2.5 KB–4 KB	Refresh noise	Programmable BTS and scheme switching
	Amortized BTS	8.2–3955.7 s (Paiva et al. 2025; Guimarães and Pereira 2025)	1024–8192	Arbitrary function	2.5 KB–4 KB	Refresh noise	Scalar product computing
Hybrid BTS	BGV/BFV/CKKS	0.94–1280 s	16,384–327,680	Arbitrary function	28 KB–6.88 MB	Refresh noise	Polynomial interpolation/polynomial approximation

$$w_i = \frac{z' - \sum_{j=0}^{i-1} 2^j w_j^{2^{i-j}} \pmod{2^{r+1}}}{2^i}$$

such that the least significant bit of w_i corresponds to the i-th bit of z'. Repeating this for r iterations extracts the r-th bit of z', thereby recovering the underlying plaintext.

5) **Slot-to-Coefficient:** Perform the discrete Fourier transform to map the refreshed slot values back into polynomial coefficients, restoring the ciphertext to its standard BGV format.

The bit extraction step constitutes the primary computational bottleneck (see Algorithm 1 for pseudocode). To mitigate the significant overhead associated with this operation, Gentry et al. proposed an optimization aimed at reducing r, and consequently, the total number of operations: first, add κ—a multiple of the modulus q satisfying $|\kappa| < B \approx q/4c_m\|s\|_1$, where c_m is the ring constant—to the preprocessed ciphertext c', so that the lowest $\log_2 B$ bits of the result are set to zero. Next, homomorphically divide the corresponding polynomial Z'' by 2^κ, reducing the number of loop iterations from r to $r - \kappa$.

Algorithm 1 Bit Extraction

Input: Ciphertext $\tilde{\boldsymbol{c}}$ encrypting constant b
Input: Ciphertext modulus q
Output: Ciphertext $\tilde{\boldsymbol{c}}'$ encrypting $b\langle 0\rangle \oplus b\langle r\rangle$

$\tilde{\boldsymbol{c}}_0 \leftarrow \tilde{\boldsymbol{c}}$
for $i = 1$ **to** r **do**
  $acc \leftarrow \tilde{\boldsymbol{c}}$
  for $j = 0$ **to** $i - 1$ **do**
    $tmp \leftarrow HomExp(\tilde{\boldsymbol{c}}_j,\ 2^{i-j})$
    $(acc \leftarrow acc - 2^j \cdot tmp) \pmod q$
  end for
  $\tilde{\boldsymbol{c}}_i \leftarrow acc \cdot ((q+1)/2)^i \pmod q$
end for
return $(\tilde{\boldsymbol{c}}_0 + \tilde{\boldsymbol{c}}_r) \pmod q$

Alperin-Sheriff and Peikert (2013) observed that the bootstrapping in Gentry et al. (2012) is of limited practical applicability, as its runtime includes super logarithmic factors and exhibits high asymptotic complexity. To address these limitations, they introduced two key innovations: (i) extraction of message carrier coefficients on high-dimensional rings, and (ii) efficient mappings between the coefficient and slot domains. Specifically, Alperin-Sheriff and Peikert (2013) first employs a homomorphic trace map to extract the desired coefficients by constructing a tower of extension rings $\mathbb{Z} = R^{(0)} \subseteq R^{(1)} \ldots \subseteq R^{(r)}$, which allows the trace function to be decomposed hierarchically as $Tr_{R/\mathbb{Z}} = Tr_{R^{(1)}/R^{(0)}} \circ \ldots \circ Tr_{R^{(r)}/R^{(r-1)}}$, reducing the complexity from quadratic to quasi-linear. They further generalize the ring switching technique of Gentry et al. (2012) to enable transitions between rings that share only a common subring, removing the need for nested relations. This decouples the plaintext coefficient ring from the multi-slot bootstrapping ring, facilitating efficient coefficient-to-slot and slot-to-coefficient conversions.

HElib (Halevi and Shoup 2021, 2018) integrates multiple bootstrapping optimizations, achieving significant improvements in both runtime efficiency and noise management robustness. While retaining the BGV bootstrapping procedure, HElib contributed two major enhancements. First, the supported plaintext modulus space is extended to any prime power $q = p^e + 1$. They showed how to perform modular reduction by decomposing $z \in \mathbb{Z}_q$ into two base-p digital segments: $[z]_q = \left(z\langle r-1,\ldots,0\rangle_p - z\langle e+r-1,\ldots,e\rangle_p\right) \pmod{p^r}$, thereby removing the restriction $p = 2$. A family of univariate polynomials F_e is then constructed such that for any prime p and $e \geq 1$, if $z \equiv z_0 \pmod{p^e}$, then $F_e(z) = z_0 \pmod{p^{e+1}}$. By replacing the original squaring step with F_e, the iterative digit-extraction generalizes to

$$w_i = \frac{z - \sum_{j=0}^{i-1} p^j \cdot F_e^{i-j}(w_j)}{p^i}$$

Second, building on the trace-map optimizations of Alperin-Sheriff and Peikert (2013), HElib applies tensor decomposition to implement the linear map between coefficient and slot domains. The $\phi(m)$-dimensional transform is decomposed into subproblems of dimension $\max_i \phi(m_i)$. Instead of performing computationally expensive ring-switching, HElib embeds the plaintext ring $R_{p^r} = \mathbb{Z}_{p^r}[X]/(\Phi_m(X))$ into a hypercube, where each axis corresponds to an m_i. The inverse mapping is then performed via a sequence of polynomial evaluations—one per axis—therefore avoiding both the coarse-grained modulus decomposition and the high-dimensional ring operation costs of the traditional approach.

Bootstrapping in BFV scheme

In BFV, the message is multiplied by a scaling factor during encryption and stored in the most significant bits of the ciphertext. Exploiting this property, Fan and Vercauteren (2012) pointed out that provided the noise remains below a certain threshold, directly truncating the lower bits of the ciphertext does not affect decryption correctness. Compared to BGV's bit-by-bit extraction,

truncation in BFV brings a significant efficiency advantage. For the scalar product between the truncated ciphertext and the secret key, Fan and Vercauteren (2012) proposed a binary matrix representation that eliminates zero entries and leverages cascaded additions to accelerate computation.

The BFV bootstrapping procedure proceeds as follows:

1) **Modulus switching:** Convert the ciphertext modulus from q to 2^n via $[2^n \cdot ct/q]$, where $n = \lfloor \log_2 q \rfloor$. This prepares the ciphertext for more efficient modular reduction in subsequent calculations.
2) **Low-bit truncation:** For each coefficient of $ct[0]$ and $ct[1]$, retain the S_R most significant bits, i.e. $d_i = (ct[i] \mod 2^n) \gg (n - S_R), \quad i = 0, 1,$ where S_R is derived from the noise upper bound, satisfying $\|e_i\| < \Delta/\nu$ and $S_R = \lceil \nu \cdot t \rceil$ to ensure the residual noise manageable.
3) **Homomorphic scalar product:** Represent the constant terms of polynomial $d_0(x)$ and $s_j \cdot x^j \cdot d_1(x)$ for $j = 0, ..., N-1$ into a $(N+1) \times S_R$ binary matrix. Expanding each integer into its binary form, the dot product is simplified to summing the matrix rows, which can be implemented using a carry-save adder reduction: repeatedly combine three rows into two by computing bit-wise sums $a_i \oplus b_i \oplus c_i$ and carries $\bar{a}_{i-1}\bar{b}_{i-1} \oplus \bar{c}_{i-1}\bar{b}_{i-1} \oplus \bar{a}_{i-1}\bar{c}_{i-1}$, with indices -1 and $S_R + 1$ predefined as 0. Once only two rows remain, a standard bitwise carry adder yields the result $w \in [0, 2^{S_R})$.
4) **Modular reduction:** Recover the message via

$$\left[\left[\frac{t \cdot [2^{n-S_R} \cdot w]_{2^n}}{2^n}\right]\right]_t$$

Chen and Han (2018) replaced the traditional bit-by-bit extraction polynomial with a low-order "least significant digit removal" polynomial $f(x) \equiv (x - (x \mod p)) \mod p^e$. This scheme can remove ν lowest bits of an e-bit integer in base p with a single operation, reducing the circuit depth of homomorphic digit extraction from $(e-1)\log p$ to $\nu \log p + \log e$, significantly optimizing the bootstrapping efficiency of FV/BGV. Work (Chen and Han 2018) further proposed "Slim Bootstrapping", which limits the required digit extraction operations to a single execution for application scenarios requiring only $\mathbb{Z}_{p^r}^k$ plaintext space rather than its d-dimensional extended ring. This optimization achieves an approximate d-fold speedup compared to a full bootstrapping.

Geelen and Vercauteren (2023) demonstrated that the homomorphic decryption processes of BFV and BGV satisfy identical noise boundary conditions, thereby unifying their bootstrapping algorithms under a single framework and proving equivalence in time complexity. This corrects the earlier conclusion in Chen and Han (2018) that BFV bootstrapping is inherently more efficient than BGV. In addition, they resolved an error in the linear transformation optimization under cyclotomic fields in Chen and Han (2018), eliminating potential bootstrapping failures for certain parameter sets.

Bootstrapping in CKKS scheme

In word-wise FHE schemes, BGV/BFV perform modular reduction via digit extraction over finite rings, realizing an exact residue computation at the cost of bit or radix-level decomposition and relatively deep arithmetic circuits. By contrast, CKKS implements modular reduction by homomorphically approximating the modular reduction function with a polynomial, achieving faster computational efficiency.

Based on the approximate modulo methods, the CKKS bootstrapping algorithms can be divided into Best uniform approximation in the L_∞ norm and Best approximation in the L_2 norm.

Infinity-norm-based approximation methods

Infinity-norm methods seek to minimize the maximum approximation error. Typically they first approximate a periodic proxy for the mod function (e.g. a sine/cosine function) and then find a polynomial that fits this smooth function with minimal L_∞ error. Key sub-approaches include Taylor series expansion, Chebyshev interpolation, and minimax polynomial design via the Remez algorithm.

Taylor Series Expansion

In 2018, Cheon et al. (2018) designed the first bootstrapping algorithm tailored to the CKKS scheme, leveraging its intrinsic approximate computation properties. The core insight is that the modular reduction function $F(t) = [t]_q$ approximates the identity function $y = x$ in the neighborhood of zero and exhibits oscillatory characteristics with period q, which highly coincides with the local linearity and periodicity of trigonometric functions. Based on this observation, the following approximate equation is established:

$$[\langle ct, sk\rangle]_q = \frac{q}{2\pi} \sin\left(\frac{2\pi}{q}\langle ct, sk\rangle\right) + O(\epsilon^3 \cdot q)$$

where $\epsilon \cdot q \geq |[\langle ct, sk\rangle]_q|$. To prevent the exponential growth in circuit depth caused by high-order Taylor expansion, Cheon et al. proposed an iterative expansion strategy based on double-angle formula. This method begins by computing the initial value within a small angle range using Euler's formula $\exp(i\theta) = \cos\theta + i \cdot \sin\theta$,

and then progressively doubles the angle through repeated squaring $\exp(2i\theta) = (\exp(i\theta))^2$, until the target trigonometric value is accurately approximated. By combining trigonometric approximation with iterative local expansion, this scheme reduces bootstrapping complexity to be linear in the depth of the decryption circuit, achieving breakthrough optimization.

Following this design idea, Jutla and Manohar (2022) achieved arbitrary-precision bootstrapping: they first approximated the modulo function with a sine series, then approximated the sine function itself through the Taylor series of e^{ix}, reconstructing high-order terms of the sine series via a sequence of squaring operations. Jutla et al. proved that within a small interval of width ε around the modulo point, the approximation error of the positive term series of order n for the modulo function is $O(\varepsilon^{2n+1})$. Since the sine-series coefficients have norm less than 2, the entire polynomial chain needs only to be evaluated at a precision marginally greater than $(2n+1)\log(1/\varepsilon)$, which matches the precision required for the approximation. This work achieved 100-bit precision bootstrapping and improved performance in lower-precision scenarios.

Chebyshev Interpolation

Although the Taylor-series approach pioneered CKKS bootstrapping, it requires $O(\log q)$ multiplicative depth (Chen et al. 2019), and its complexity of coefficient-slot transforms grows linearly with the number of plaintext slots, so it is not scalable for densely-packed ciphertexts and is suboptimal in level consumption.

Subsequent research showed that replacing the Taylor series with a Chebyshev interpolating polynomial yields much better uniform approximation with lower degree. Chebyshev polynomials minimize the L_∞ error almost optimally for a given degree. In 2019, Chen et al. (2019) first adopted Chebyshev interpolation for the sine-based mod reduction. Instead of centering a series at 0, they construct the degree-n Chebyshev interpolant $P_n(x) = \sum_{k=0}^{n} c_k T_k(x)$ that approximates $\frac{q}{2\pi}\sin\left(\frac{2\pi}{q}t\right)$ on the target interval. This yields dramatically smaller max-error for the same degree compared to Taylor polynomials. Additionally, they used a dynamic programming approach from HElib (Halevi and Shoup 2021) to decide the optimal level collapsing strategy for a generic multi-leveled linear transforms, resulting in a large increase on both computational efficiency and throughput.

Han and Ki (2020) observed that in previous bootstrapping the input to the trigonometric step can be written as $t = qI + m$ with $m \ll q$, so that t/q lies very close to an integer. Works (Cheon et al. 2018; Chen et al. 2019) approximate $\frac{q}{2\pi}\sin\left(\frac{2\pi}{q}t\right)$ by a polynomial that is accurate `globally` on a wide interval, not fully utilizing this structural property. In contrast, Han and Ki approximate $\cos(2\pi t)$ only on small intervals around `specific points` after an appropriate scaling and shift (to drop q), and combine the polynomial approximation with the cosine double-angle formula. This optimization yields a better polynomial $p(t)$ with smaller degree and lower approximation error for the inputs actually arising in bootstrapping. Moreover, they proposed a generalized key switching algorithm for the Full-RNS (Residue Number Systems) variant of CKKS, reducing computational complexity while improving accuracy.

Bossuat et al. (2021) observed that the aforementioned CKKS bootstrapping algorithms rely on sparse secret keys to reduce circuit depth, which fails to provide 128-bit security under attack models specifically targeting sparse RLWE keys. To overcome this limitation, they first proposed a general RNS-CKKS approximate rescaling algorithm capable of homomorphically evaluate arbitrary d-dimensional polynomial functions with optimal depth $\lceil\log(d+1)\rceil$. This method maintains consistent ciphertext scales in homomorphic addition by backtracking scaling factors at each level, thereby eliminating errors from scale mismatches. Second, they designed a double-hoisting encrypted matrix–vector multiplication algorithm. This algorithm exploits precomputation and reuse of polynomial decompositions in the inner loops, merges the inner `ModDown` operation into the outer loops, and employs a mixed decomposition base strategy. Together, these techniques significantly reduce the number of NTT (Number Theoretic Transform) and CRT (Chinese Remainder Theorem) extension operations, cutting key-switching costs by nearly half. Finally, Bossuat et al. incorporated these optimizations into the bootstrapping procedure, achieving high-precision CKKS bootstrapping at the 128-bit security level.

Remez Algorithm (Minimax Approximation)

The Remez algorithm is an iterative method to directly compute the minimax optimal polynomial approximation for a given function. In CKKS bootstrapping, after recognizing the importance of uniform error, researchers turned to Remez to obtain the optimal L_∞ polynomial for the modular reduction. In 2021, Lee et al. (2021) proposed an improved multi-interval Remez algorithm, which constructs minimax optimal approximation polynomials for arbitrary continuous functions over unions of finite intervals. Additionally, to overcome the approximation region limitations of the arcsine function in modular reduction, they embedded it into a composite structure of sine/cosine functions. This function-composition mapping effectively extends the approximation domain of the modular reduction function, allowing arcsine approximation polynomials to be combined with periodic functions, thereby improving bootstrapping accuracy. This work

increased CKKS bootstrapping accuracy by 5.4–10.2 bits, significantly outperforming traditional piecewise polynomial approximation schemes. Notably, Remez yields the lowest possible maximum error for a given degree, outperforming Chebyshev interpolants slightly in accuracy. In essence, the Remez-designed polynomial is the best infinity-norm approximation, guaranteeing that the maximum error is minimized and equioscillates over the approximation domain.

L_2-norm-based approximation methods

In contrast to uniform-error methods, L_2-norm-based approaches minimize the mean squared error of the approximation. Rather than ensuring the worst-case error is as small as possible, these methods aim to minimize the overall error energy. For the CKKS bootstrapping context, an L_2 fit can sometimes achieve a lower polynomial degree for a given average accuracy, accepting that the maximum error might be slightly larger than a minimax solution.

Lee et al. (2020) pointed out that when using Infinity-norm-based approximation methods, the trigonometric function dominates the approximation error, which limits the message size and thus plaintext precision. Instead, they proposed to approximate the mod function more directly by finding a polynomial $P(t)$ that minimizes $\int |P(t) - [t]_q|^2 dt$ over the domain. Using the Chebyshev polynomials as a basis, they achieved a lower approximation error with a lower degree compared with the best method (Han and Ki 2020) at that time. Another benefit was a relaxed parameter constraint: earlier schemes required a large bootstrapping modulus Q relative to the message (scaling like $O(m^{3/2})$) to mitigate error, whereas the low-error L_2 polynomial allowed $O(m)$, saving level consumption in bootstrapping.

Comparison and analysis

BGV and BFV are exact integer schemes whose bootstrapping procedures refresh the available evaluation depth while simultaneously resetting the noise level. In BFV, digit extraction is used to clear low-order noise bits; while in the BGV algorithm, it removes the overflow term appears during modulus switching. Consequently, the bootstrapped ciphertext recovers the exact original plaintext and the noise is reduced to enable subsequent computations. By contrast, CKKS is an approximate scheme whose precision is measured in retained bits: its bootstrapping—essentially a deep homomorphic evaluation circuit—inevitably introduces approximation error, resulting in a loss of precision. Unlike exact schemes, CKKS bootstrapping restores the ciphertext modulus and evaluation capacity but does not eliminate the noise; instead, the bootstrapping-induced error is accumulated into the message. Therefore, the primary objective of CKKS bootstrapping is to maximize the precision preserved after the refresh procedure.

This objective is intrinsically linked to the degree of the polynomial approximations employed during bootstrapping: higher-degree polynomials yield lower approximation errors and higher output precision, but at the expense of increased multiplicative depth and computational complexity. In practice, the trade-off is managed by selecting the minimal polynomial degree that satisfies the target precision requirements within the plaintext domain. Approximations measured in the infinity norm offer stronger worst-case precision guarantees: Remez polynomials ensure the error is uniformly bounded over the target interval, while Chebyshev yields a slightly larger error but offers a computationally simpler alternative, which still achieves near-optimal worst-case performance. By contrast, L_2-norm-based approximations optimize average case accuracy and can exhibit larger errors at worst-case points than L_∞-norm-based methods.

Regarding parameter selection, most implementations set the cyclotomic polynomial degree N to a power of two, which facilitates an efficient radix-2 NTT and convenient negacyclic modulus $X^N + 1$. The ciphertext modulus Q is determined by required multiplicative depth and bootstrapping overhead; in implementations it is realized as a RNS modulus chain $Q = \prod_{i=1}^{l} q_i$. While increasing $|Q|$ (i.e., adding primes to the chain) expands the noise budget to support higher-precision bootstrapping, it incurs costs in terms of larger key sizes and higher computational complexity. Another critical parameter in CKKS is the scaling factor Δ (typically a power of two), which encodes real values into integers and dictates message precision. Bootstrapping will reset the ciphertext to a target scale, commonly the initial Δ. A comparative summary is given in Table 4.

Bit-wise FHE bootstrapping algorithms

In 2013, Gentry et al. (2013) introduced the GSW scheme to overcome the dimension expansion and nonlinear noise growth in word-wise fully homomorphic encryption. Leveraging approximate eigenvector encryption, it replaces the tensor product of vector ciphertexts with matrix multiplication. This shift eliminates the need for modulus switching and relinearization, while bounding error growth from homomorphic multiplication within a small polynomial. This advancement established the matrix-based paradigm for homomorphic encryption.

Table 4 Comparison of approaches to word-wise bootstrapping

Scheme	BGV	BFV	CKKS			
Work flow	ModRaise - CoeffToSlot - EvalMod - SlotToCoeff - ModDown					
EvalMod	Digit extraction		L_∞-norm			L_2-norm
			Taylor	Chebyshev	Remez	
Noise	Refreshed		Accumulated			
Precision	Strictly restore message precision		Low	Near-optimal	Optimal	Near-optimal
Poly degree	Digit extraction rounds		High	Moderate	Moderate	Slightly lower than L_∞-norm methods

Gate bootstrapping

FHEW/AP bootstrapping

In 2014, Brakerski and Vaikuntanathan (2014) applied Barrington's theorem (Barrington 1986) to convert depth-d NAND gate circuits into length-4^d branching programs. Exploiting the noise asymmetry property of GSW, they constructed a homomorphic decryption circuit with polynomially bounded noise growth. In the same year, Alperin-Sheriff and Peikert (2014) proposed a more efficient bootstrapping approach—known as AP bootstrapping—that achieves slower error accumulation. Rather than constructing Boolean circuits, AP bootstrapping models homomorphic decryption as an arithmetic function: a scalar product of keys and ciphertexts, followed by rounding. By mapping messages into symmetric groups, AP bootstrapping facilitates a more direct bootstrapping process.

In 2015, the FHEW scheme (Ducas and Micciancio 2015) implemented AP bootstrapping over rings, decreasing ciphertext refresh time to under one second. Its key innovation is encoding cyclic group elements into the group of roots of unity via $i \mapsto X^i$. This allows a single ciphertext to represent cyclic group elements, significantly improving storage overhead and computational efficiency compared with the ciphertext vector representation in the AP scheme (Alperin-Sheriff and Peikert 2014). In FHEW, the bootstrapping ciphertext takes the form $E_z(m) = [\boldsymbol{a}, \boldsymbol{a} \cdot z + \boldsymbol{e}] + u \cdot x^m G$, where z denotes the secret key, u is an invertible element close to $Q/(2t)$ in $\mathbb{Z}_Q$, and G is the gadget matrix with adjustable base $\beta \geq 2$ to balance the trade-off between norm constraints and matrix size.

FHEW bootstrapping can be described in three main steps:

1) **Initialization:** Accumulator $ACC \leftarrow u \cdot X^{b+q/4} G$, which corresponds to a noise-free encryption of ciphertext component b;
2) **Homomorphic accumulation:** The scalar product is accumulated at the exponent position by multiplying $X^{-a_i s_i}$. After completion, the exponent becomes

$$b + \frac{q}{4} - \sum a_i s_i = \frac{q}{2} m + e + \frac{q}{4} \tag{1}$$

Denote equation (1) as φ.

3) **Extract the most significant bit:** For correctness, $|e| < q/4$ is required, so exponents less than $q/2$ corresponds to message $m = 0$, otherwise $m = 1$ (where $q = 2N$). However, direct comparison is infeasible in homomorphic computation, as data remains encrypted. To address this, a test vector is constructed: $test = 1 + x + x^2 + ... + x^{N-1}$. $X^\varphi \cdot test$ equals:

$$\begin{cases} x + x^2 + \ldots + x^{N-1} - 1, & \varphi = 1 \\ x^2 + x^3 + \ldots + x^{N-1} - 1 - x, & \varphi = 2 \\ \ldots \\ -1 - x - x^2 - \ldots - x^{N-1}, & \varphi = N \\ -x - x^2 - \ldots - x^{N-1} + 1, & \varphi = N+1 \\ \ldots \end{cases}$$

$$\Rightarrow \text{constant term const} = \begin{cases} -1, & \varphi \leq q/2 \Rightarrow m = 0 \\ 1, & \varphi > q/2 \Rightarrow m = 1 \end{cases}$$

Once the constant term is determined, the message can be recovered via a simple homomorphic linear transformation $m = \frac{\text{const}+1}{2}$, hence completing the homomorphic decryption process.

TFHE/GINX bootstrapping

Building on FHEW, Gama et al. (2016) refined the AP scheme (Alperin-Sheriff and Peikert 2014) with GINX bootstrapping, which computes the scalar product using MUX gates. GINX bootstrapping substantially decreases the size of bootstrapping keys but restricts support to binary key distributions, whereas AP bootstrapping handles arbitrary key distributions. The TFHE scheme (Chillotti et al. 2020) enhanced FHEW and implemented GINX bootstrapping over the torus, cutting gate bootstrapping time from 690 ms to 13 ms on a single core and shrinking the bootstrapping key size from 1GB to 16MB at the same security level.

TFHE's critical improvement is replacing the "internal product" between RGSW ciphertexts with an "external

product" between RLWE and RGSW ciphertexts. This substitution transforms matrix–matrix multiplication into a matrix–vector multiplication, effectively eliminating one dimension and sharply diminishing both computational redundancy and memory requirements. The central computation in this process is "blind rotation", functionally equivalent to the homomorphic accumulation step in FHEW bootstrapping, i.e., rotating by multiplying the monomial X^t where t is encrypted on the torus. As the most computationally expensive step in bootstrapping, blind rotation typically accounts for over 80% of the total bootstrapping time (Zama 2022).

The blind rotation is implemented through CMUX gates. Each CMUX takes three inputs: selection factor b (1 bit, RGSW ciphertext), and two options d_0 and d_1 (RLWE ciphertexts), with the output result calculated as $d_b = b \cdot (d_1 - d_0) + d_0$, where d_b remains encrypted as an RLWE-type ciphertext.

The accumulator ACC is initialized as the RLWE encryption of $test \cdot X^{-b}$. The secret key s is binary decomposed, and each bit is used as the selection factor in CMUX; the other two inputs respectively encrypt ACC and $ACC \cdot X^{a_i}$, reflecting the two possible outcomes:

$$ACC \cdot X^{s_i \cdot a_i} = \begin{cases} ACC, & s_i = 0 \\ ACC \cdot X^{a_i}, & s_i = 1 \end{cases}$$

Algorithm 2 TFHE/GINX Bootstrapping

Input: LWE ciphertext $\boldsymbol{c} = (a_1, \ldots, a_n, b) \in \mathbb{T}_q^{n+1}$
Input: Bootstrapping key $BSK = (BSK_1, \ldots, BSK_n)$
Input: Key-switching key $KSK = \left\{KSK_{(i,j)}\right\}$
Input: Look-up table $V \in \mathbb{T}_N[X]^{(k+1)}$
Output: LWE ciphertext $\boldsymbol{c}'' \in \mathbb{T}_q^{n+1}$

1: $ModSwitch(\boldsymbol{c}) = \tilde{\boldsymbol{c}} = \left(\tilde{a}_1, \ \ldots, \ \tilde{a}_n, \ \tilde{b}\right)$ // Modulus Switching
2: $ACC_0 \leftarrow V \cdot X^{-b}$ // Blind Rotation
3: **for** $i = 1$ **to** n **do**
4: $\quad ACC_i \leftarrow BSK_i \boxdot \left(X^{\tilde{a}_i} \cdot ACC_{i-1} - ACC_{i-1}\right) + ACC_{i-1}$ // External Product
5: **end for**
6: $\boldsymbol{c}' = \left(a'_1, \ \ldots, \ a'_n, \ b'\right) \leftarrow SampleExtract(ACC_n)$ // Sample Extraction
7: $\boldsymbol{c}'' = \left(0, \ \ldots, \ b'\right) - \sum_{i=1}^{kN} \sum_{j=1}^{l} \left(a'_i\right)_j \cdot KSK_{(i,j)}$ // Key Switching
8: **return** $\boldsymbol{c}''$

After processing all bits, the final blind rotation output is $V \cdot X^{-b+\sum a_i s_i} = VX^{-(\Delta m+e)}$, where V is a precomputed lookup table containing the mapping $\Delta m + e \mapsto m$. At this stage, the coefficients of V are rotated by $\Delta m + e$ positions so that the mapping corresponding to m is aligned at the constant position, allowing sample extraction to recover the message. The pseudocode of GINX bootstrapping is shown in Algorithm 2 (in TFHE (Chillotti et al. 2020), the variant omitting the final key switching step is referred to as TLWE-to-TLWE bootstrapping).

Both AP and GINX bootstrapping described above are identity bootstrapping processes. To evaluate logic gates, a linear mapping must be applied before bootstrapping. Taking the AND gate as an example: message 0 is encoded as $-q/8$, while 1 corresponds to $q/8$. The lookup table is defined as $V = \sum_{j=0}^{N-1} \frac{q}{8} X^j$, which functions similarly to a sign function—returning $q/8$ when the message is positive and $-q/8$ when negative. The linear mapping for AND is $\mu_1 + \mu_2 - q/8$, producing a positive result only if both inputs are 1. Bootstrapping then decreases the error, yielding a low-noise ciphertext. Other Boolean gates follow the same principle, differing only in their linear mapping formulas. Importantly, the output of gate bootstrapping retains the same encoding format as the input, permitting direct chaining into the next gate without extra transformation and therefore ensuring computational continuity.

Gate bootstrapping optimization

In 2018, Zhou et al. (2018) reduced the number of serial external products in blind rotation by half via computing two key bits at once, and thereby lowering the homomorphic refresh time to under 10 ms. This technique, known as "Blind Rotation Unfolding", was further refined in Bourse et al. (2018). Although blind rotation unfolding increases overall computational load and memory consumption, it shortens the length of the critical path in the bootstrapping procedure, allowing full utilization of hardware parallelism. For example, MOSFHET (Guimarães et al. 2024) applied this method

to optimize multi-value bootstrapping and achieved satisfactory performance.

In 2021, Micciancio and Polyakov (2021) proposed a unified framework to compare the TFHE and FHEW schemes. Experimental analysis showed that the main performance gap between the two primarily stems from the distinction between AP bootstrapping and GINX bootstrapping, noting that TFHE's optimization of replacing internal products with external products can be adapted to FHEW. This work extended the key distribution of GINX bootstrapping from binary to ternary, and demonstrated that the TFHE/GINX achieves better performance with binary and ternary keys, while FHEW/AP is more suitable for Gaussian-distributed keys. Building on this, Joye and Paillier (2022) generalized GINX bootstrapping to support keys with arbitrary radix representation and combined previous techniques (Zhou et al. 2018; Bourse et al. 2018) to reduce the number of external products to n/d, where n is the key length and d is the number of bits processed per iteration.

Algorithm 3 Automorphism-based Blind Rotation Algorithm (Lee et al. 2023)

Input: $ACC = RLWE_z^0\left(f'\left(X\right)\right), f'\left(X\right) = f\left(X^{-g}\right)\cdot X^{-g\beta}$
Input: $\vec{\alpha} = \left\{\alpha_i = \pm g^{k_i} \pmod{2N} | i = 0,\ \ldots,\ n-1\right\}$
Input: $I_l^+ = \left\{i : \alpha_i = g^l\right\},\ I_l^- = \left\{i : \alpha_i = -g^l\right\}$, where $l \in [0,\ N/2-1]$ and g is generator
Input: Bootstrapping key $BSK = (BSK_0,\ \ldots,\ BSK_{n-1})$
Input: Automorphism key $\{ak_{g^u}\}_{u\in[1,w]},\ ak_{-g}$
Output: $ACC = RLWE_z\left(f'\left(X^{-g^{(N/2)-1}}\right)\cdot X^{\sum \alpha_i s_i}\right)$

$v \leftarrow 0$
for $l = N/2-1$ **to** 1 **do**
  for $j \in I_l^-$ **do**
    $ACC \leftarrow ACC \boxdot BSK_j$
  end for
  $v \leftarrow v+1$
  if $\left(I_{l-1}^- \neq \oslash$ **or** $v = w$ ***or*** $l = 1\right)$ **then**
    $ACC \leftarrow Auto_{g^v}\left(ACC,\ ak_{g^v}\right)$
    $v \leftarrow 0$
  end if
end for
for $j \in I_0^-$ **do**
  $ACC \leftarrow ACC \boxdot BSK_j$
end for
$ACC \leftarrow Auto_{-g}\left(ACC,\ ak_{-g}\right)$
for $l = N/2-1$ **to** 1 **do**
  for $j \in I_l^+$ **do**
    $ACC \leftarrow ACC \boxdot BSK_j$
  end for
  $v \leftarrow v+1$
  if $\left(I_{l-1}^+ \neq \oslash$ **or** $v = w$ **or** $l = 1\right)$ **then**
    $ACC \leftarrow Auto_{g^v}\left(ACC,\ ak_{g^v}\right)$
    $v \leftarrow 0$
  end if
end for
for $j \in I_0^+$ **do**
  $ACC \leftarrow ACC \boxdot BSK_j$
end for
return ACC

More recently, work (Lee et al. 2023) combined the advantages of AP bootstrapping and GINX bootstrapping. Similar to Bonnoron et al. (2017), Lee et al. (2023) used ring automorphisms to realize blind rotation, but worked over a power-of-two cyclotomic ring rather than a prime-order cyclic ring. This choice complicates the implementation of even-power automorphisms in the absence of modular inverses, yet leads to more practical parameter settings. This scheme supports arbitrary key distributions while maintaining a small key size, surpassing previous schemes in both memory requirements and runtime. The pseudocode of its core blind rotation algorithm is provided in Algorithm 3.

Belorgey et al. (2024) built upon the approach of KLSS (Kim et al. 2023), which utilizes a twin-gadget decomposition of the ciphertext to streamline external products within the keyswitch process. This reduces the cost of modular computations and FFTs by further decoupling large integer decomposition from cyclotomic ring arithmetic. Concretely, Mariya et al. replaced the CRT-based decomposition with a radix-2^k large integer decomposition and reinterpret ring polynomials as bivariate polynomials $F(X, Y)$, where the auxiliary variable Y encodes the radix factor used in the gadget decomposition ($Y = 2^{-k}$). The bivariate polynomial representation constrains coefficient norms, consequently improving the efficiency of polynomial multiplication in homomorphic operations. In addition, the authors report that these changes cut the number of FFTs by one-eighth and linear arithmetic work by one-sixth in TFHE gate bootstrapping, bringing bootstrapping CPU latency below 5.5 ms in their implementation. Importantly, the optimization is not TFHE-specific, which yields significant speedups for word-wise schemes such as CKKS and BFV as well.

Programmable bootstrapping

Bootstrapping in bit-wise fully homomorphic encryption not only serves to suppress noise and refresh ciphertexts, but can also "customize" the information extracted by configuring the coefficients of the test vector, i.e. programmable bootstrapping. This process enables the homomorphic evaluation of functions while lowering the noise by encoding function values into the test vector.

Multi-value programmable bootstrapping was first proposed in Carpov et al. (2019), allowing the computation of multiple function values on the same input through a single bootstrapping operation, with no restrictions on function types (aside from requiring negative cyclic properties). The key idea is to encode each function as a lookup table polynomial P_i and construct a shared polynomial Q. By decomposing P_i into $Q \cdot P'_i$, only a single blind rotation using Q is required, after which constant multiplication by P'_i yields each target value. To limit noise growth in output ciphertexts, each factor P'_i should have coefficients of small magnitude. Concretely, the squared norm of this polynomial is given by $\|TestVec\|_2^2 \leq s \cdot (q-1)^2$. The 6-to-6 lookup table implemented by this algorithm runs in 1.6 s under 128-bit security level, nearly ten times faster than scheme (Bonnoron et al. 2017) at 100-bit security level.

Guimarães et al. (2021) proposed two programmable bootstrapping combination strategies: tree-based and chain-based, which aims at accelerating the evaluation of large lookup tables and high-precision functions. In high-precision settings, this work decreases the required number of bootstrapping calls in Carpov et al. (2019) and changes the asymptotic growth of the error variance in the output radix r from $O(r^2)$ to $O(r)$. The chain-based PBS introduced in Guimarães et al. (2021) was later generalized in Clet et al. (2022) to enable the evaluation of arbitrary functions by expanding the plaintext space. Building on the decomposition concept from Guimarães et al. (2021), Bergerat et al. (2023) developed a general modular reduction algorithm applicable to arbitrary bases, and introduced a hybrid ciphertext format combining CRT-based and radix representation. They further proposed a new bootstrapping algorithm that supports computation over single or multiple ciphertexts and can efficiently evaluates general lookup tables in radix, CRT, or hybrid formats.

Liu et al. (2022) focused on improving the evaluation accuracy of sign function. They decomposed high-precision ciphertexts into digits, then sequentially performed homomorphic floor functions starting from the least significant bit, gradually reducing the ciphertext modulus Q proportionally until it is small enough to support efficient bootstrapping. Based on this approach, Liu et al. designed two homomorphic algorithms for the floor function and extended them to arbitrary function evaluation in 4-bit plaintext spaces. In addition, they proposed a homomorphic digit decomposition method that effectively expands the supported plaintext message space.

Chillotti et al. (2021) proposed a generalized PBS capable of evaluating multiple functions without introducing additional computational overhead or noise. This work provides an in-depth analysis of noise growth during tensor product relinearization, and derives optimal parameter configurations for the TFHE framework. This enables multiplication between LWE ciphertext without bootstrapping, which leads to the design of a padding-free programmable bootstrapping algorithm (WoP-PBS) and eliminates the traditional bootstrapping's requirement for prior knowledge of the message's most significant bit or the negative cyclicity that univariate functions must satisfy. To address the negative cyclicity limitation, subsequent

research (Clet et al. 2022; Kluczniak and Schild 2021; Yang et al. 2021) generally adopts the function decomposition strategy: splitting an arbitrary functions f into several subfunctions f_i that meet structural restrictions, and then performing Full-Domain Functional Bootstrap (FDFB) by homomorphically computing each f_i.

Guimarães et al. (2024) systematically summarized key techniques for TFHE performance optimization and noise reduction. Subsequently, by combining and innovating upon some of these techniques, they proposed two significant improvements. First, they introduced multi-value bootstrapping with blind rotation unrolling, which supports the evaluation of encrypted lookup tables without format restrictions, and the number of evaluated LUTs is independent of the failure probability. Second, they proposed seed expansion for RLWE. By storing only the pseudorandom seed used to generate $\boldsymbol{a}$ in RLWE samples, $\boldsymbol{a}$ can be dynamically generated on demand, thus reducing RLWE storage requirements by 50%. Furthermore, in Guimarães et al. (2024), the memory loading of the $\boldsymbol{b}$ component was interleaved with the computation of $\boldsymbol{a}$, fully exploiting CPU instruction-level parallelism to mitigate the latency overhead introduced by the memory optimization. The team also released an open-source software library integrating key operators—such as programmable bootstrapping, circuit bootstrapping, and key switching—alongside optimization algorithms including Full-Domain Functional Bootstrapping, Blind Rotation Unrolling, Multi-value Bootstrapping, and Automorphism-based Bootstrapping, hence laying a solid foundation for systematic study of bit-wise scheme optimization.

Bon et al. (2024) argue that the LWE-ciphertext multiplication introduced in Chillotti et al. (2021) is prohibitively expensive in practice and, in response, present a novel framework for Boolean function evaluation. Their scheme introduces an intermediate homomorphic layer before encryption, mapping bits to the ring $\mathbb{Z}_p$ using disjoint subsets $\varepsilon_b(0),\ \varepsilon_b(1) \in \mathbb{Z}_p$ to encode 0 and 1, respectively. During homomorphic computation, the underlying sets $\varepsilon_b(0)$ and $\varepsilon_b(1)$ may shift, expand, or shrink, but they never overlap, ensuring no information loss. By computing the mapping ε, the framework compresses the evaluation of multi-input Boolean functions into a single homomorphic addition followed by a bootstrapping, significantly improving evaluation efficiency. They further provided methods for constructing effective mappings and heuristic algorithms for decomposing Boolean circuits into Boolean functions optimized for this approach. Experiments evaluated common cryptographic primitives—including SIMON, Trivium, Keccak permutation, Ascon S-box, and AES S-box—demonstrate promising performance gains at 128-bit security level with an error rate of 2^{-40}.

Table 5 Summary of programmable bootstrapping optimizations

Work	Full domain	Large LUT	Multi value	Time (ms)	Key size (MB)	Noise growth	BTS number
Bonnoron et al. (2017)	×	✓	×	6400	9420.8	$O(n^{4.5}\sigma)$*	1
Carpov et al. (2019)	×	✓	✓	1500	8192†	$O\left(\sqrt{nkN\left(l\beta^2\sigma^2+\epsilon^2\right)}\right)$‡	1
Guimarães et al. (2021)	×	✓	×	378.2	4403.2	$O\left(\sqrt{nkN\left(l\beta^2\sigma^2+\epsilon^2\right)}\right)$	$\lceil \log_\beta m \rceil$§
Chillotti et al. (2021)	✓	✓	✓	266.4¶	–	$O\left(\sqrt{nkNl\beta^2\sigma^2}\right)$	1 or 3‖
Yang et al. (2021)	✓	✓	×	–	3079.17	$O\left(\sqrt{\frac{q^2}{Q^2}nlN\beta^2\sigma_{bk}^2}\right)$	2
Liu et al. (2022)	✓	✓	×	442	–	$O\left(\frac{q}{Q_{ks}}\sqrt{nN\log Q}\right)$	2
Clet et al. (2022)	✓	✓	×	119.4	–	$O\left(\sqrt{nlN(\sigma_{KS}^2+k\beta^2\sigma_{BK}^2)}\right)$	≤ 4
Kluczniak and Schild (2021)	✓	✓	×	13,781	4624.31	$O\left(\sqrt{\frac{q^2}{Q^2}nNl\beta^2\sigma^2}\right)$	1
Guimarães et al. (2024)	✓	✓	✓	76.53††	23808	–	1
Bon et al. (2024)	✓	✓	×	9.5	–	–	1

* σ is the error parameter in the key

† The original text is "multi-value bootstrapping key < 2 GB and the switching key ≈ 6 GB"

‡ $\epsilon = 1/2\beta^l$

§ m is the original message integer

¶ There is no execution time data in the original paper. The data here comes from the implementation in Guimarães et al. (2024)

‖ "1" comes from the improved Gen-PBS that can evaluate multiple LUTs, and "3" is from WoP-PBS that overcomes the limitation of needing to know the most significant bit of the message

†† The article contains many algorithms, and the data here comes from a global function bootstrap with original optimization technology

Table 5 summarizes the characteristics, computational complexity, and noise growth of the aforementioned optimization techniques. Beyond theoretical breakthroughs in lattice-based schemes, research has also explored porting bit-wise bootstrapping algorithms to other FHE constructions to leverage their unique strengths. Kim et al. (2023) integrated blind rotation into the bootstrapping procedures of BGV, BFV, and CKKS. This not only enhanced precision but also, for the first time, mitigated the CKKS vulnerability identified in Li and Micciancio (2021). Furthermore, their algorithm supports arbitrary plaintext spaces and operations in small rings. Building on the automorphism-based blind rotation from Lee et al. (2023), Xiang et al. (2023) implemented an NTRU-based variant, achieving a 2.7× smaller key size and 3× faster computation. Li et al. (2024) also worked under the NTRU assumption. They employed approximate gadget decomposition to diminish the decomposition length and the number of polynomial multiplications. By combining this with the key expansion technique from Bourse et al. (2018), they halved the number of NTTs required in blind rotation and further reduced automorphism computations to $3n/5$ by merging consecutive empty and symmetric sets, achieving performance that comprehensively surpasses (Xiang et al. 2023).

Due to its ability to support arbitrary non-linear operations, programmable bootstrapping demonstrates broad applicability in privacy-preserving computation. Chillotti et al. (2021) proposed a universal evaluation framework for deep neural networks based on FHE. Utilizing PBS and leveled homomorphic operations to evaluate complex functions, their method overcomes neural network depth limitations without requiring modifications to the original model. Similarly, a team from Korea University (Yoo et al. 2024) integrated PBS into a cloud-based location privacy protection system to efficiently handle non-linear retrieval conditions for geographical locations (such as radial range queries), solving the problem of predicate evaluation in ciphertext space. In another application, Trama et al. (2025) employed PBS to build an 8-bit FHE processor abstraction that executes encrypted data operations in the form of an assembly instruction set. Their system achieves performance improvements exceeding 20% on simple tasks such as average computation, sum of squares, and sigmoid evaluation. These practices indicate that PBS is steadily evolving into a pivotal technology bridging homomorphic encryption theory and practical applications, providing a scalable and functionally complete universal paradigm for privacy-enhanced computing.

Circuit bootstrapping

Circuit bootstrapping originates from the TFHE framework, which provides two homomorphic implementation modes: FHE and LHE (Leveled Homomorphic Encryption). The FHE mode adheres to the bit-wise design principles, performing low-latency gate bootstrapping on small-precision single-bit ciphertexts after each Boolean gate, allowing evaluation of any function. In contrast, the LHE mode employs parameter sets tailored for specific circuits, and typically executes a single circuit bootstrapping after completing computation. While gate bootstrapping takes and outputs LWE ciphertexts, circuit bootstrapping accepts LWE ciphertexts as input and outputs RGSW ciphertexts. Both approaches take blind rotation, sample extraction, and key switching as core operators, but circuit bootstrapping chains multiple programmable bootstrapping (originally gate bootstrapping) procedures. It first generates LWE ciphertexts of messages corresponding to certain positions of the target RGSW ciphertext through PBS, then applies key switching to aggregate and reconstruct these ciphertexts into the desired RGSW format.

Building upon homomorphic CMUX gates, TFHE can construct Turing-complete Boolean circuits, enabling arbitrary known-depth Boolean computations, essentially equivalent to deterministic finite automata. However, a notable limitation is that CMUX control lines (RGSW ciphertexts) must be generated by the user, while data lines (RLWE ciphertexts) can be either newly created or derived from previous outputs. To support full circuit composability and remove this restriction, Chillotti et al. (2017) proposed circuit bootstrapping, realizing transformation from LWE ciphertexts to RGSW ciphertexts. This algorithm exploits the internal sparsity of RGSW ciphertexts, which comprise $(k+1)l$ RLWE ciphertexts encrypting messages $\mu \cdot h_i$ respectively, where μ is the message and h_i is the i-th row of gadget matrix. The procedure begins by obtaining LWE ciphertexts of messages $\mu\beta^{-1}, ..., \mu\beta^{-l}$ through l rounds of TLWE-to-TLWE bootstrapping, and then packs and reconstructs them into the target RGSW ciphertext using key switching. The pseudocode is shown in Algorithm 4.

Algorithm 4 Circuit Bootstrapping in TFHE

Input: LWE ciphertext $\boldsymbol{c} = (a_1, \ldots, a_n, b) \in \mathbb{T}_q^{n+1}$
Input: Bootstrapping key $BSK = (BSK_1, \ldots, BSK_n)$
Input: Key-switching key $KSK = \left\{ KSK^{(f_u)} \mid 0 \le u \le k+1 \right\}$, where $f_u(x) = -K_u \cdot x \, (u \le k), \; f_{k+1}(x) = x$
Output: RGSW ciphertext $C \in \mathbb{T}_N[X]^{(k+1)l \times (k+1)}$

for $w = 1$ **to** l **do**
  // TLWE-to-TLWE Bootstrapping
  $\boldsymbol{c}^w \leftarrow Bootstrapping_{\beta^{-w}}(\boldsymbol{c})$
  for $u = 1$ **to** $k+1$ **do**
  // Key Switching
    $\boldsymbol{c}^{(u,w)} = Keyswitch\left(KSK^{(f_u)}, \; \boldsymbol{c}^w\right)$
  end for
end for
return $C = \left(\boldsymbol{c}^{(u,w)}\right)_{1 \le u \le k+1, \; 1 \le w \le l}$

In 2021, Chillotti et al. (2021) replaced the l rounds of serial programmable bootstrapping in the original algorithm with the improved PBSmanyLUT, which evaluates multiple LUTs in a single bootstrapping while keeping the noise level unchanged.

In 2024, Wang et al. proposed a series of optimization works for circuit bootstrapping (Wang et al. 2024a, b, c). In work (Wang et al. 2024a), Wang et al. observed that the original circuit bootstrapping algorithm separately computed RLWE′(m) and RLWE′(sm), where RLWE′ denotes an RLWE ciphertext decomposed via a gadget. Each RLWE′ ciphertext requires l key switching operations, identified as the main source of computational and storage overhead in circuit bootstrapping. By introducing a scheme switching algorithm to transform RLWE′(m) into RLWE′(sm), this work successfully halved the number of key switching. In Wang et al. (2024b), they proposed a novel circuit bootstrapping framework achieving a 9.9× speedup compared over the original TFHE implementation and reducing key size by 15.6×. This improvement results from two primary strategies. First, the circuit bootstrapping workflow was redesigned: the authors replace functional bootstrapping with the multi-value bootstrapping of Carpov et al. (2019) and change the output format from LWE to RLWE so that computation stays entirely over the ring, thereby avoiding costly format conversions. Second, they adopt the automorphism-based blind-rotation algorithm and exploit the negative correlation between automorphism cost and the sparsity of the LWE coefficients a_i. By rounding the a_i to increase sparsity, they reduce the number of required automorphisms and, consequently, the overall computational expense.

The Wang et al. (2024c) approach was further accelerated by 2.6× compared to Wang et al. (2024b) through preprocessing, and the NTT-based operations were replaced with FFT to mitigate the high cost of modular arithmetic. To support this, an FFT preprocessing mechanism was developed to make the HomTrace computation compatible with FFT parameters, followed by a split FFT algorithm to reduce numerical errors. These optimizations, combined with parameter tuning, yield a 12.1× speedup over (Wang et al. 2024b) and a 3.4× improvement over TFHEpp (Matsuoka 2020). Additionally, a high-precision HomTrace algorithm based on GLWE key switching with multi-bit extraction enables a new circuit bootstrapping scheme for integer inputs.

Amortized bootstrapping

Bit-wise schemes based on Boolean circuits inherently lack support for single instruction multiple data (SIMD), with computation granularity limited to the bit level. For multi-bit message operations, multiple iterations and combinations of programmable bootstrapping are required, leading to computational overhead growing linearly with data size and severely limiting efficiency. To improve the practicality of bit-wise schemes, many studies have aimed to expand batch bootstrapping capabilities, amortizing the cost of programmable bootstrapping through parallelization.

In 2018, Miccianco and Sorrell (2018) introduced the first batch bootstrapping on FHEW framework, simultaneously refreshing $O(n)$ ciphertexts with controllable cost (degrading bootstrapping performance by $O(n^\epsilon)$ compared to FHEW, for any constant $\epsilon > 0$). The amortized cost for single message refresh is $O(3^{1/\epsilon} n^\epsilon)$,

significantly better than $O(n)$ in FHEW. The algorithm proceeds as follows: using a key switching variant from Brakerski and Vaikuntanathan (2014), $\varphi(d)$ (where d is the cyclotomic polynomial degree) high-noise LWE ciphertexts are first packed into a single RLWE ciphertext, on which bootstrapping is performed. The refreshed LWE ciphertexts are then extracted individually via sample extraction. The fundamental idea of this scheme is to bootstrap the RLWE ciphertext using FHEW's monomial exponent encoding strategy and to recursively decompose high-degree polynomial multiplication into several low-degree subpolynomial multiplications via the Nussbaumer negative wrapped convolution algorithm (Nussbaumer 1980). Unlike the standard Fast Fourier Transform (FFT), this algorithm fixes the loop variable $Y = X^{n^\epsilon}$ (rather than $Y = X^{n_i}$ for each recursion level), thereby bounding the recursion depth by $1/\epsilon$ and strictly suppressing noise explosion.

Multiple subsequent works have built on the framework of Miccianco and Sorrell (2018). Both Guimarães et al. (2023) and De Micheli et al. (2024) replaced the Nussbaumer algorithm with NTT, utilizing ring automorphisms for rotation factor multiplication, hence simplifying the bootstrapping workflow and reducing computational overhead from $2^{O(1/\epsilon)}n^{1/\epsilon}$ to $(1/\epsilon)\cdot n^{1/\epsilon}$. However, their designs differ. Guimarães et al. (2023) completes automorphism multiplication directly on RGSW ciphertexts, designing a ciphertext shrinking algorithm that combines gadget decomposition with double CRT representation to reduce ciphertext dimensions and modulus while maintaining a constant noise ratio. They further extend the plaintext space to $\mathbb{Z}_t$, overcoming the single-bit limitation, and provide the first C++ implementation of amortized bootstrapping, demonstrating its practicality. In contrast, De Micheli et al. (2024) uses RLWE registers as the basis for exponentiation and adopts prime cyclotomic ring instead of power-of-2 cyclotomic or cyclic rings, avoiding invertibility constraints on automorphisms and achieving a more compact ciphertext structure.

Paiva et al. (2025) optimized the Guimarães et al. (2023) workflow by improving its two-stage inverse NTT using incomplete NTT, enabling a broader choice of moduli. They established a trade-off mechanism between performance and Decryption Failure Rate (DFR), and explored new parameter set configurations thoroughly, achieving a 1.41× speedup compared to Guimarães et al. (2023) under 8-bit plaintexts while reducing DFR from 2^{-17} to 2^{-39}.

Guimarães and Pereira (2025) proposed a new method for homomorphic computation of monomial and polynomial multiplications to optimize the blind rotation step in RLWE bootstrapping. Unlike traditional approaches that require traversing the bootstrapping key, this algorithm directly multiplies and sums non-zero terms. This work extended the algorithm to support bootstrapping keys with non-binary distributions. By rewriting the computation expressions in a nested form, the method requires only $O(hN\log N)$ RGSW-RLWE external products to complete bootstrapping, where h is the hamming weight of the bootstrapping key, therefore significantly decreasing parameters and the size of evaluation keys. Building on this, Guimarães et al. introduced RLWE packing with an amortized cost of $O(h)$, whose C implementation achieved 3.9–41.5× speedup over TFHE-rs (Zama 2022) while reducing the bootstrapping key size by 50.4×.

In a different line of work, Liu and Wang (2023a), Liu and Wang (2023b) departed from the RLWE bootstrapping paradigm and introduced tensor ring structures, achieving an amortized cost of $\tilde{O}(1)$ homomorphic multiplications. They first proposed a batch processing framework compatible with FHEW/TFHE by employing tensor rings $R = R_1 \otimes R_2 \otimes R_3$ composed of three complementary rings for encoding. Input vectors are alternately packed into the $R_1 \otimes R_2$ and $R_1 \otimes R_3$ spaces, while cross terms arising from multiplications are eliminated using dual bases. Individual components of dot products are extracted through trace computation. By alternating between tensors and traces, this framework avoids dimension explosion caused by introducing more rings, enabling multi-input batch dot products in constant-depth circuits. Within this framework, n LWE ciphertexts can be bootstrapped simultaneously with an amortized cost of $\tilde{O}(\lambda^{0.75})$. In the follow-up work (Liu and Wang 2023b), the authors designed new homomorphic ring multiplication and inverse DFT transformations, and integrated them with RLWE bootstrapping from Miccianco and Sorrell (2018) to further reduce the amortized cost to $\tilde{O}(1)$.

Bootstrapping algorithms in hybrid FHE schemes

Since word-wise schemes naturally support SIMD batch processing, many studies have implemented batch bootstrapping and LUT evaluation for bit-wise schemes through word-wise constructions, mapping bit operations to parallel arithmetic operations and achieving significant breakthroughs in PBS throughput and cross-operation compatibility, thereby enabling Boolean-Arithmetic hybrid computation.

BGV/BFV-based methods achieve accurate evaluation of Boolean gates and nonlinear functions through polynomial interpolation. In 2023, Liu and Wang (2023c) proposed a batch gate bootstrapping scheme based on BGV/BFV, which supports parallel evaluation of heterogeneous Boolean gates. This work further integrates

programmable bootstrapping to keep the amortized homomorphic multiplication cost at $\tilde{O}(1)$. The refreshed LWE ciphertexts are obtained by combining slot-to-coefficient conversion from word-wise schemes with sample extraction technique from bit-wise schemes. Experimental results shows that Boolean gate computation takes under 5 ms, and function evaluation time under 9-bit plaintext modulus is approximately 6.7 ms. In subsequent work (Liu and Wang 2024), Liu et al. defined a relaxed BFV bootstrapping that ensures correctness only for predefined subsets of the plaintext space instead of the entire message domain. This relaxation reduces the polynomial degree and allows optimal parameters selection for a given plaintext modulus, facilitating function embedding within BFV bootstrapping. For binary gate evaluation, this method achieves roughly 3× speedup over (Liu and Wang 2023c).

Lee et al. (2024) developed functional bootstrapping for FV/CKKS schemes, with outputs fixed as FV ciphertexts. The FV-to-FV bootstrapping supports plaintext modulus switching, expanding the evaluation domain of functions; while the CKKS-to-FV bootstrapping incorporates a format conversion module. The essential idea of this scheme is to compute general lookup tables through linear combinations of step functions over the commutative ring $\mathbb{Z}_{p^r}$ (where p is prime). However, efficiency is limited due to the small number of FV bootstrapping slots: 9-bit precision sign function evaluation takes approximately 46.5 s, with amortized cost far higher than works (Liu and Wang 2023c, 2024).

Similarly, Drucker et al. (2024) implemented integer computation in the CKKS scheme using interpolation. This algorithm employs step functions generated by high-degree polynomial composition as "clean-up" tools, truncating decimal places to recover accurate exact integers, thereby enabling efficient unbounded binary circuit evaluation. To handle deep integer circuit computation, integers are decomposed into binary form, reducing errors in binary coefficients, and the results are recombined into low-error ciphertexts. Chung et al. (2024) extended the aforementioned approach of modeling binary gate computations with real interpolation polynomials, thereby enabling CKKS to support both binary operations and small integer arithmetic. They encode message m as $\exp(2i\pi m/t)$ based on t-th roots of unity, securely evaluating multi-input-output look-up tables using low-degree multivariate polynomials. This algorithm is compatible with all word-wise fully homomorphic encryption schemes (BGV/BFV/CKKS) and features multi-level parallelism. First, multiple indices embedded in a multivariate polynomial enable the parallel evaluation of several inputs within a single LUT. Second, multiple slots inside a ciphertext permit concurrent evaluation of multiple LUTs. Third, reusing ciphertexts across computation rounds allows different LUTs to be evaluated with minimal additional overhead. However, as the scheme is not based on functional bootstrapping, it lacks the ability to reduce error, and still requires additional CKKS or BFV bootstrapping in deep computations, differing functionally from LUT-evaluating bootstrapping schemes.

Bae et al. (2024) designed two bootstrapping algorithms for bit vector ciphertexts in the CKKS scheme: BinBoot can refresh single ciphertext, while GateBoot evaluates Boolean gates on two ciphertexts. Their approach is compatible with FHEW/TFHE ciphertext formats and can construct new bit-wise gate bootstrapping algorithms combined with ring packing technique (Bae et al. 2023), achieving full-slot operations (i.e., utilizing all available CKKS slots, corresponding to $N/2$ for a ring dimension N). Alexandru et al. (2024) developed a general functional bootstrapping based on CKKS, supporting evaluation of arbitrary functions over $\mathbb{Z}_p$ ($p \geq 2$). They use triangular Hermite interpolation that satisfies $f(x_i) = y_i$ and $f'(x_i) = 0$ at interpolation points to suppress noise. By deriving general Fourier-series expressions from triangular interpolation theory, Alexandru et al. enable arbitrary function evaluation—from this perspective, Bae et al. (2024) can be regarded as the special case of this scheme when $p = 2$. Alexandru et al. (2024) also designed multi-precision sign function evaluation in $\mathbb{Z}_p$ and generalized error reduction methods via polynomial interpolation. Implemented in OpenFHE (Badawi et al. 2022), this method achieves an amortized time of 0.72 ms for 8-bit look-up tables.

Bae et al. (2024) further addressed bottlenecks in small-integer processing and functional bootstrapping for CKKS with two new algorithms. SI-BTS (Small Integer Bootstrapping) refreshes low-bit integer ciphertexts, supports arbitrary look-up tables at negligible cost, and allows multi-function evaluation on a single plaintext more efficiently than repeated bootstrapping. This algorithm is also compatible with FHEW/TFHE formats, allowing batch processing of bit-wise ciphertexts. BB-BTS (Batch-Bits Bootstrapping) extends it by processing multiple CKKS ciphertexts in parallel, functioning as a throughput-optimized variant of BinBoot in Bae et al. (2024). Kim and Noh (2024) adopted this computational framework, and integrated iterative techniques with modulus switching to implement an arbitrary-precision modular reduction for CKKS.

More recently, Kim (2025) proposed a new CKKS-based paradigm for homomorphic unsigned integer computation. By representing a large integer (e.g., 64 bits) as a combination of small-bit-width vectors (e.g., 4 bits), batch integer operations are efficiently implemented utilizing CKKS slot parallelism. This scheme integrates

Table 6 Performance summary of amortized bootstrapping schemes

Paradigm	References	Base scheme	Amortized cost	Amortized time (ms)	Noise growth
Bit-wise	Miccianco and Sorrell 2018	FHEW	$O(3^{1/\epsilon}\, n^{\epsilon})^{*}$	–	$\widetilde{O}(n^{2+3/\epsilon})$
	Guimarães et al. 2023	FHEW/GSW	$O(n^{\epsilon}/\epsilon)$	1205	$\widetilde{O}(n^{1+1/\epsilon})$
	De Micheli et al. (2024)	FHEW	n^{ϵ}/ϵ	–	Poly(λ)
	Liu and Wang (2023a)	FHEW/TFHE	$\widetilde{O}(\lambda^{0.75})$	–	$O(nrqN\sqrt{N\log QE})^{\dagger}$
	Liu and Wang (2023b)	FHEW	$\widetilde{O}(1)$	–	Poly(λ)
	Guimarães and Pereira (2025)	GSW	$O(h)^{\ddagger}$	2.5	$O(\sqrt{h}\,\lambda\log\lambda)$
	Paiva et al. (2025)	FHEW/TFHE	$O(n^{\epsilon}/\epsilon)$	584.7	$\widetilde{O}(\lambda^{1.5+\lg N/\alpha})$
Word-wise	Liu and Wang (2023c)	BGV/FV	$\widetilde{O}(1)$	6.7	$\Theta(\lambda^{\log(\lambda)})$
	Liu and Wang (2024)	BFV	–	1.3	SuperPoly(λ)
	Drucker et al. (2024)	CKKS	–	0.0277[§]	SuperPoly(λ)
	Lee et al. (2024)	FV/CKKS	–	2956	SuperPoly(λ)
	Bae et al. (2024)	CKKS	–	0.036[§]	SuperPoly(λ)
	Alexandru et al. (2024)	CKKS	–	0.72	SuperPoly(λ)
	Bae et al. (2024)	CKKS	–	3.75	SuperPoly(λ)
	Kim (2025)	CKKS	–	12	SuperPoly(λ)

[*] The notation ϵ denotes any positive constant

[†] r is the number of ciphertexts to be packaged. Other parameters are shown in Table 2

[‡] h denotes the hamming weight of the LWE secret key

[§] The entries marked with § correspond to *gate* bootstrapping latency; while other reported numbers are *functional* bootstrapping times

arithmetic operations, comparison operations, and bootstrapping, constructing a complete set of fully homomorphic computation primitives.

Table 6 summarizes the comparison of amortized bootstrapping algorithms built on bit-wise schemes and hybrid bootstrapping based on word-wise schemes, considering amortized cost, actual runtime, and noise growth. Although amortized bootstrapping schemes based on word-wise generally lack analysis of amortized costs, their execution speeds are 2–3 orders of magnitude faster than those of bit-wise schemes. However, their performance comes at the expense of superpolynomial noise growth and substantially larger parameter sizes compared to bit-wise approaches. Additionally, "programmable bootstrapping" in word-wise schemes and integer computation in CKKS essentially rely on polynomial interpolation or approximation techniques to emulate nonlinear operations, differing from the flexible function construction based on gate circuits in bit-wise schemes. This distinction originates from differences in underlying design paradigms, with no fundamental breakthrough yet achieved.

Future research

Despite significant progress in FHE operations and bootstrapping, its large-scale application still faces three principal challenges: heavy computational and storage overhead, difficulty of cross-scheme collaboration, and weak multi-party scalability. Future work can address these challenges through the following directions:

1) **Co-optimization between software and hardware:** Overcoming efficiency bottlenecks requires close integration of algorithmic design and hardware architecture. On the algorithm side, it is necessary to continuously optimize the complexity of key operations such as high-precision functional bootstrapping and batch programmable bootstrapping, exploring more possible implementation methods. On the hardware side, deep mining of parallel computing potential in mature platforms such as GPU/FPGA, designing dedicated instruction sets and memory architectures, and exploring heterogeneous computing efficiency with new hardware paradigms such as Processing in Memory (PIM) can yield substantial gains. Software-hardware co-design will maximize resource utilization and deliver order-of-magnitude performance improvements.
2) **Cross-scheme interoperability and standardization:** Current different FHE schemes (word-wise BGV/BFV/CKKS vs. bit-wise FHEW/TFHE) exhibit significant differences in parameter systems, ciphertext structures, and applicable scenarios, hindering technology integration and ecosystem con-

struction. Consequently, there is an urgent need to establish unified API interfaces, security parameter frameworks, and ciphertext conversion standards. Existing scheme switching algorithms such as Chimera (Boura et al. 2020) and PEGASUS (Lu et al. 2021) still have much room for efficiency improvement. Promoting standardization will not only simplify collaboration between bit-wise and word-wise schemes, balancing parallel computing and arbitrary function evaluation, but also lower development thresholds and accelerate industrial implementation.

3) **Extension to Practical Multi-Party Computation Models:** Real-world applications often involve multiple data owners, where the single-client–server model proves inadequate. Multi-Key FHE (MKFHE) supports collaborative computation on ciphertexts from multiple parties, while Threshold FHE (ThFHE) enables distributed key management with built-in fault tolerance mechanisms. Bootstrapping remains central to sustaining deep computations in these settings, yet faces amplified challenges: MKFHE demands efficient cross-party key-switching, whereas ThFHE necessitates secure partial-decryption protocols to prevent leakage of sensitive intermediates. A common challenge in multi-party FHE settings is that partial decryption outputs may leak information about the secret key or plaintext. A classical defense is noise flooding, which adds large random noise to mask such leakage, but this requires very large parameters and leads to substantial computational and communication overhead. Recent noise sanitization techniques (e.g., smudging Dai et al. 2025) achieve equivalent indistinguishability guarantees by carefully sampling or mildly perturbing the noise using refined probabilistic analysis, thereby significantly reducing the required noise budget and parameter sizes. Overcoming these obstacles would expand FHE′s applicability, ensuring data sovereignty and privacy security in distributed environments, thereby providing fundamental support for secure multi-party computation.

Conclusion

This survey systematically reviews the technical evolution and optimization pathways of bootstrapping algorithms for fully homomorphic encryption. By deeply analyzing the three principal paradigms—word-wise, bit-wise, and hybrid—we expose the fundamental distinctions and performance trade-offs among them. Word-wise schemes, built on the RLWE assumption, confront their main bootstrapping challenge in nonlinear modular reduction. They offer substantial SIMD parallelism but suffer from super-polynomial noise growth and large ciphertext sizes. Bit-wise schemes excel in low-latency gate bootstrapping and programmable bootstrapping capable of evaluating arbitrary functions. This paradigm exhibits slow noise growth and only requires polynomial-level hardness assumptions, yet lacks native batching ability. Hybrid approaches merge word-wise SIMD with batched LUT evaluation, attaining orders-of-magnitude throughput gains. However, their reliance on polynomial interpolation or approximation sets them apart from the zero-overhead table lookup realizations available in bit-wise designs.

Despite huge advances, contemporary FHE research still faces three main bottlenecks: limited computational efficiency, a fragmented ecosystem of incompatible scheme families, and insufficient support for realistic application models especially multi-party computation. Future work should emphasize hardware–software co-design, advance cross-scheme interoperability, and explore novel computation models to move FHE from theoretical progress toward practical, verifiable, and highly parallel privacy-preserving deployments.

Acknowledgements
The authors gratefully acknowledge the computing resources provided by the Key Laboratory of Cyberspace Security Defense at the Institute of Information Engineering, CAS.

Author contributions
Tianyu Li performed a systematic classification of FHE bootstrapping algorithms, conducted an in-depth analysis of the advantages and limitations of the different technical approaches, and drafted the manuscript. Zhiwei Wang, Lutan Zhao, Ruida Wang and Qingyun Niu reviewed the manuscript and provided critical feedback and revision suggestions. Xianhui Lu, Dan Meng and Rui Hou have gone through the article. All authors read and approved the final version of the manuscript.

Funding
This work was supported by the National Key Research and Development Program of China(2023YFB4503200), the Strategic Priority Research Program of the Chinese Academy of Sciences(XDB0690100), and the National Natural Science Foundation of China(621252208, 62202464).

Data availability
This manuscript has not been published and is not under consideration for publication elsewhere.

Declarations

Conflict of interest
The authors declare no conflict of interest.

Received: 20 August 2025 Accepted: 2 March 2026
Published online: 07 April 2026

References

Al Badawi A, Polyakov Y (2023) Demystifying bootstrapping in fully homomorphic encryption. Cryptology eprint archive

Alexandru A, Kim A, Polyakov Y (2024) General functional bootstrapping using ckks. Cryptology ePrint Archive
Alperin-Sheriff J, Peikert C (2013) Practical bootstrapping in quasilinear time. In: Annual cryptology conference, pp 1–20. Springer
Alperin-Sheriff J, Peikert C (2014) Faster bootstrapping with polynomial error. In: Annual cryptology conference, pp 297–314. Springer
Anggriane SM, Nasution SM, Azmi F (2016) Advanced e-voting system using paillier homomorphic encryption algorithm. In: 2016 International conference on informatics and computing (ICIC), pp 338–342. IEEE
Badawi AA, Alexandru A, Bates J, Bergamaschi F, Cousins DB, Erabelli S, Genise N, Halevi S, Hunt H, Kim A, Lee Y, Liu Z, Micciancio D, Pascoe C, Polyakov Y, Quah I, RV S, Rohloff K, Saylor J, Suponitsky D, Triplett M, Vaikuntanathan V, Zucca V (2022) OpenFHE: open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915. https://eprint.iacr.org/2022/915
Bae Y, Cheon JH, Kim J, Park JH, Stehlé D (2023) Hermes: efficient ring packing using mlwe ciphertexts and application to transciphering. In: Annual international cryptology conference, pp 37–69. Springer
Bae Y, Cheon JH, Kim J, Stehlé D (2024) Bootstrapping bits with CKKS. In: Annual international conference on the theory and applications of cryptographic techniques, pp 94–123. Springer
Bae Y, Kim J, Stehlé D, Suvanto E (2024) Bootstrapping small integers with CKKS. In: international conference on the theory and application of cryptology and information security, pp 330–360. Springer
Barrington DA (1986) Bounded-width polynomial-size branching programs recognize exactly those languages in NC. In: Proceedings of the eighteenth annual ACM symposium on theory of computing, pp 1–5
Belorgey MG, Carpov S, Gama N, Guasch S, Jetchev D (2024) Revisiting key decomposition techniques for FHE: simpler, faster and more generic. In: International conference on the theory and application of cryptology and information security, pp 176–207. Springer
Bergerat L, Boudi A, Bourgerie Q, Chillotti I, Ligier D, Orfila J-B, Tap S (2023) Parameter optimization and larger precision for (T) FHE. J Cryptol 36(3):28
Bon N, Pointcheval D, Rivain M (2024) Optimized homomorphic evaluation of boolean functions. IACR Trans Cryptogr Hardw Embed Syst 2024(3):302–341
Boneh D, Goh E-J, Nissim K (2005) Evaluating 2-DNF formulas on ciphertexts. In: Theory of cryptography conference, pp 325–341. Springer
Bonnoron G, Ducas L, Fillinger M (2017) Large FHE gates from tensored homomorphic accumulator. Cryptology ePrint Archive, Paper 2017/996. https://eprint.iacr.org/2017/996
Bossuat J-P, Mouchet C, Troncoso-Pastoriza J, Hubaux J-P (2021) Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In: Annual international conference on the theory and applications of cryptographic techniques, pp 587–617. Springer
Boura C, Gama N, Georgieva M, Jetchev D (2020) Chimera: combining ring-lwe-based fully homomorphic encryption schemes. J Math Cryptol 14(1):316–338
Bourse F, Minelli M, Minihold M, Paillier P (2018) Fast homomorphic evaluation of deep discretized neural networks. In: Annual international cryptology conference, pp 483–512. Springer
Brakerski Z (2012) Fully homomorphic encryption without modulus switching from classical GapSVP. In: Annual cryptology conference, pp 868–886. Springer
Brakerski Z, Vaikuntanathan V (2014) Efficient fully homomorphic encryption from (standard) LWE. SIAM J Comput 43(2):831–871
Brakerski Z, Gentry C, Vaikuntanathan V (2014) (leveled) fully homomorphic encryption without bootstrapping. ACM Trans Comput Theory (TOCT) 6(3):1–36
Brakerski Z, Vaikuntanathan V (2011) Fully homomorphic encryption from ring-lwe and security for key dependent messages. In: Annual cryptology conference, pp 505–524. Springer
Brakerski Z, Vaikuntanathan V (2014) Lattice-based FHE as secure as PKE. In: Proceedings of the 5th conference on innovations in theoretical computer science, pp 1–12
Carpov S, Izabachène M, Mollimard V (2019) New techniques for multi-value input homomorphic evaluation and applications. In: Cryptographers' track at the RSA conference, pp 106–126. Springer
Chen H, Chillotti I, Song Y (2019) Improved bootstrapping for approximate homomorphic encryption. In: Annual international conference on the theory and applications of cryptographic techniques, pp 34–54. Springer
Chen H, Han K (2018) Homomorphic lower digits removal and improved FHE bootstrapping. In: Annual international conference on the theory and applications of cryptographic techniques, pp 315–337. Springer
Cheon JH, Han K, Kim A, Kim M, Song Y (2018) Bootstrapping for approximate homomorphic encryption. In: Annual international conference on the theory and applications of cryptographic techniques, pp 360–384. Springer
Cheon JH, Kim A, Kim M, Song Y (2017) Homomorphic encryption for arithmetic of approximate numbers. In: International conference on the theory and application of cryptology and information security, pp 409–437. Springer
Chillotti I (2022) TFHE Deep Dive – Part I – Ciphertext types. Zama Blog. https://www.zama.ai/post/tfhe-deep-dive-part-1
Chillotti I, Gama N, Georgieva M, Izabachène M (2020) TFHE: fast fully homomorphic encryption over the torus. J Cryptol 33(1):34–91
Chillotti I, Gama N, Georgieva M, Izabachène M (2017) Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In: International conference on the theory and application of cryptology and information security, pp 377–408. Springer
Chillotti I, Joye M, Paillier P (2021) Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In: International symposium on cyber security cryptography and machine learning, pp 1–19. Springer
Chillotti I, Ligier D, Orfila J-B, Tap S (2021) Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In: International Conference on the theory and application of cryptology and information security, pp 670–699. Springer
Chung H, Kim H, Kim Y-S, Lee Y (2024) Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption. Cryptology ePrint Archive
Clet P-E, Zuber M, Boudguiga A, Sirdey R, Gouy-Pailler C (2022) Putting up the Swiss army knife of homomorphic calculations by means of TFHE functional bootstrapping. Cryptology ePrint Archive
Dai X, Wu W, Feng Y (2025) Multikey fully homomorphic encryption: removing noise flooding in distributed decryption via the smudging lemma on discrete Gaussian distribution. IET Inf Secur 2025(1):7550044. https://doi.org/10.1049/ise2/7550044
De Micheli G, Kim D, Micciancio D, Suhl A (2024) Faster amortized FHEW bootstrapping using ring automorphisms. In: IACR international conference on public-key cryptography, pp 322–353. Springer
Drucker N, Moshkowich G, Pelleg T, Shaul H (2024) Bleach: cleaning errors in discrete computations over CKKS. J Cryptol 37(1):3
Ducas L, Micciancio D (2015) FHEW: bootstrapping homomorphic encryption in less than a second. In: Annual international conference on the theory and applications of cryptographic techniques, pp 617–640. Springer
Fang H, Qian Q (2021) Privacy preserving machine learning with homomorphic encryption and federated learning. Future Internet 13(4):94
Fan J, Vercauteren F (2012) Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive
Gama N, Izabachène M, Nguyen PQ, Xie X (2016) Structural lattice reduction: generalized worst-case to average-case reductions and homomorphic cryptosystems. In: Annual international conference on the theory and applications of cryptographic techniques, pp 528–558. Springer
Geelen R, Vercauteren F (2023) Bootstrapping for BGV and BFV revisited. J Cryptol 36(2):12
Gentry C (2009) Fully homomorphic encryption using ideal lattices. In: Proceedings of the forty-first annual ACM symposium on theory of computing, pp 169–178
Gentry C, Halevi S (2011) Implementing gentry's fully-homomorphic encryption scheme. In: Annual international conference on the theory and applications of cryptographic techniques, pp 129–148. Springer
Gentry C, Halevi S (2019) Compressible FHE with applications to pir. In: Theory of cryptography conference, pp 438–464. Springer
Gentry C, Halevi S, Peikert C, Smart NP (2012) Ring switching in BGV-style homomorphic encryption. In: International conference on security and cryptography for networks, pp 19–37. Springer
Gentry C, Halevi S, Smart NP (2012) Better bootstrapping in fully homomorphic encryption. In: International workshop on public key cryptography, pp 1–16. Springer

Gentry C, Sahai A, Waters B (2013) Homomorphic encryption from learning with errors: conceptually-simpler, asymptotically-faster, attribute-based. In: Annual cryptology conference, pp 75–92. Springer
Gilad-Bachrach R, Dowlin N, Laine K, Lauter K, Naehrig M, Wernsing J (2016) Cryptonets: applying neural networks to encrypted data with high throughput and accuracy. In: International conference on machine learning, pp 201–210. PMLR
Guimarães A, Borin E, Aranha DF (2024) Mosfhet: optimized software for FHE over the torus. J Cryptogr Eng 14(3):577–593
Guimarães A, Borin E, Aranha DF (2021) Revisiting the functional bootstrap in TFHE. IACR Trans Cryptogr Hardw Embedd Syst 229–253
Guimarães A, Pereira HV (2025) Fast amortized bootstrapping with small keys and polynomial noise overhead. Cryptology ePrint Archive
Guimarães A, Pereira HV, Van Leeuwen B (2023) Amortized bootstrapping revisited: simpler, asymptotically-faster, implemented. In: International conference on the theory and application of cryptology and information security, pp 3–35. Springer
Halevi S, Shoup V (2021) Bootstrapping for HElib. J Cryptol 34(1):7
Halevi S, Shoup V (2018) Faster homomorphic linear transformations in HElib. In: Annual international cryptology conference, pp 93–120. Springer
Han K, Ki D (2020) Better bootstrapping for approximate homomorphic encryption. In: Cryptographers' track at the RSA conference, pp 364–390. Springer
HomomorphicEncryption.org (2018) Homomorphic Encryption Standard. https://homomorphicencryption.org/standard/. Version 2.0
Joye M, Paillier P (2022) Blind rotation in fully homomorphic encryption with extended keys. In: International symposium on cyber security, cryptology, and machine learning, pp 1–18. Springer
Jutla CS, Manohar N (2022) Sine series approximation of the mod function for bootstrapping of approximate he. In: Annual international conference on the theory and applications of cryptographic techniques, pp 491–520. Springer
Kim J (2025) Efficient homomorphic integer computer from CKKS. IACR Trans Cryptogr Hardw Embed Syst. https://doi.org/10.46586/tches.v2025.i4.873-898
Kim M, Song Y, Li B, Micciancio D (2020) Semi-parallel logistic regression for GWAS on encrypted data. BMC Med Genomics 13(Suppl 7):99
Kim A, Deryabin M, Eom J, Choi R, Lee Y, Ghang W, Yoo D (2023) General bootstrapping approach for RLWE-based homomorphic encryption. IEEE Trans Comput 73(1):86–96
Kim M, Lee D, Seo J, Song Y (2023) Accelerating he operations from key decomposition technique. In: Annual international cryptology conference, pp 70–92. Springer
Kim J, Noh T (2024) Modular reduction in CKKS. Cryptology ePrint Archive
Kluczniak K, Schild L (2021) Fdfb: full domain functional bootstrapping towards practical fully homomorphic encryption. arXiv preprint arXiv:2109.02731
Lee J-W, Lee E, Lee Y, Kim Y-S, No J-S (2021) High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In: Annual international conference on the theory and applications of cryptographic techniques, pp 618–647. Springer
Lee Y, Lee J-W, Kim Y-S, No J-S (2020) Near-optimal polynomial for modulus reduction using l2-norm for approximate homomorphic encryption. IEEE Access 8:144321–144330
Lee Y, Micciancio D, Kim A, Choi R, Deryabin M, Eom J, Yoo D (2023) Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. In: Annual international conference on the theory and applications of cryptographic techniques, pp 227–256. Springer
Lee D, Min S, Song Y (2024) Functional bootstrapping for packed ciphertexts via homomorphic LUT evaluation. Cryptology ePrint Archive
Li Z, Lu X, Wang Z, Wang R, Liu Y, Zheng Y, Zhao L, Wang K, Hou R (2024) Faster NTRU-based bootstrapping in less than 4 ms. IACR Trans Cryptogr Hardw Embed Syst 2024(3):418–451
Li B, Micciancio D (2021) On the security of homomorphic encryption on approximate numbers. In: Annual international conference on the theory and applications of cryptographic techniques, pp 648–677. Springer
Liu F-H, Wang H (2023) Batch bootstrapping I:-a new framework for SIMD bootstrapping in polynomial modulus. Eurocrpyt
Liu F-H, Wang H (2023) Batch bootstrapping II:-bootstrapping in polynomial modulus only requires o (1) FHE multiplications in amortization. Eurocrpyt
Liu Z, Micciancio D, Polyakov Y (2022) Large-precision homomorphic sign evaluation using FHEW/TFHE bootstrapping. In: International conference on the theory and application of cryptology and information security, pp 130–160. Springer
Liu Z, Wang Y (2023) Amortized functional bootstrapping in less than 7 ms, with o (1) polynomial multiplications. In: International conference on the theory and application of cryptology and information security, pp 101–132. Springer
Liu Z, Wang Y (2024) Relaxed functional bootstrapping: a new perspective on BGV/BFV bootstrapping. In: International conference on the theory and application of cryptology and information security, pp 208–240. Springer
Lu W-j, Huang Z, Hong C, Ma Y, Qu H (2021) Pegasus: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In: 2021 IEEE symposium on security and privacy (SP), pp 1057–1073. IEEE
Matsuoka K (2020) TFHEpp: pure C++ implementation of TFHE cryptosystem. https://github.com/virtualsecureplatform/TFHEpp
Micciancio D, Polyakov Y (2021) Bootstrapping in FHEW-like cryptosystems. In: Proceedings of the 9th on workshop on encrypted computing & applied homomorphic cryptography, pp 17–28
Micciancio D, Sorrell J (2018) Ring packing and amortized FHEW bootstrapping. In: 45th international colloquium on automata, languages, and programming (ICALP 2018), pp 100–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik
Najjar MA, Huang R-Y, Samaraweera D, Shekhar P (2025) Secure distributed learning for cavs: defending against gradient leakage with leveled homomorphic encryption. arXiv preprint arXiv:2506.07894
Nussbaumer H (1980) Fast polynomial transform methods for multidimensional DFTS. In: ICASSP'80. IEEE international conference on acoustics, speech, and signal processing, vol. 5, pp 235–237. IEEE
Ohba Y, Sanuki T, Gravel C, Mihara K, Wakasugi A, Adachi K (2025) An NVME-based secure computing platform with FPGA-based TFHE accelerator. IEEE Access. https://doi.org/10.1109/ACCESS.2025.3561728
Paillier P (1999) Public-key cryptosystems based on composite degree residuosity classes. In: International conference on the theory and applications of cryptographic techniques, pp 223–238. Springer
Paiva TB, De Micheli G, Hafiz SM, Simplicio Jr MA, Yildiz B (2025) Faster amortized bootstrapping using the incomplete NTT for free. Cryptology ePrint Archive
Rivest RL, Adleman L, Dertouzos ML et al (1978) On data banks and privacy homomorphisms. Found Secur Comput 4(11):169–180
Rivest RL, Shamir A, Adleman L (1978) A method for obtaining digital signatures and public-key cryptosystems. Commun ACM 21(2):120–126
Samardzic N, Sanchez D (2024) Bitpacker: enabling high arithmetic efficiency in fully homomorphic encryption accelerators. In: Proceedings of the 29th ACM international conference on architectural support for programming languages and operating systems, Volume 2, pp 137–150
SEAL M (2023) Microsoft SEAL. https://github.com/microsoft/SEAL. Microsoft Research, Redmond, WA
Trama D, Boudguiga A, Clet P-E, Sirdey R, Ye N (2025) Designing a general-purpose 8-bit (t) FHE processor abstraction. IACR Trans Cryptogr Hardw Embed Syst 2025(2):535–578
Wang R, Ha J, Shen X, Lu X, Chen C, Wang K, Lee J (2024) Refined TFHE leveled homomorphic evaluation and its application. Cryptology ePrint Archive
Wang R, Wei B, Li Z, Lu X, Wang K (2024) TFHE bootstrapping: faster, smaller and time-space trade-offs. In: Australasian conference on information security and privacy, pp 196–216. Springer
Wang R, Wen Y, Li Z, Lu X, Wei B, Liu K, Wang K (2024) Circuit bootstrapping: faster and smaller. In: Annual international conference on the theory and applications of cryptographic techniques, pp 342–372. Springer
Xiang B, Zhang J, Deng Y, Dai Y, Feng D (2023) Fast blind rotation for bootstrapping FHEs. In: Annual international cryptology conference, pp 3–36. Springer
Yang Z, Järvinen K (2018) The death and rebirth of privacy-preserving wifi fingerprint localization with paillier encryption. In: IEEE INFOCOM

2018-IEEE conference on computer communications, pp 1223–1231. IEEE
Yang Z, Xie X, Shen H, Chen S, Zhou J (2021) TOTA: fully homomorphic encryption with smaller parameters and stronger security. Cryptology ePrint Archive
Yiran D, Jiang Z, Binwu X, Yi D (2024) Overview on the research status and development route of fully homomorphic encryption technology. J Electron Inf Technol 46(5):1774–1789
Yoo JS, Hong MY, Heo JW, Lee KH, Yoon JW (2024) Fast private location-based information retrieval over the torus. In: 2024 IEEE international conference on advanced video and signal based surveillance (AVSS), pp 1–7. IEEE
Zama (2022) Concrete ML: a Privacy-preserving machine learning library using fully homomorphic encryption for data scientists. https://github.com/zama-ai/concrete-ml
Zama (2022) TFHE-rs: A pure rust implementation of the TFHE scheme for Boolean and integer arithmetics over encrypted data. https://github.com/zama-ai/tfhe-rs
Zhang X, Fu A, Wang H, Zhou C, Chen Z (2020) A privacy-preserving and verifiable federated learning scheme. In: ICC 2020-2020 IEEE international conference on communications (ICC), pp 1–6. IEEE
Zhou T, Yang X, Liu L, Zhang W, Li N (2018) Faster bootstrapping with multiple addends. IEEE Access 6:49868–49876

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.